

Agentic Workflow Harness Workshop

Develop an app using sprint planning, a web Kanban board, handoff session resumes, reviews, and gated closeout.

Prerequisites to be set up

1. Visual Studio Code from Microsoft
2. **Claude Ext.** code setup in VS Code via Extensions (side-panel)



3. **Claude Code CLI** installed via Winget

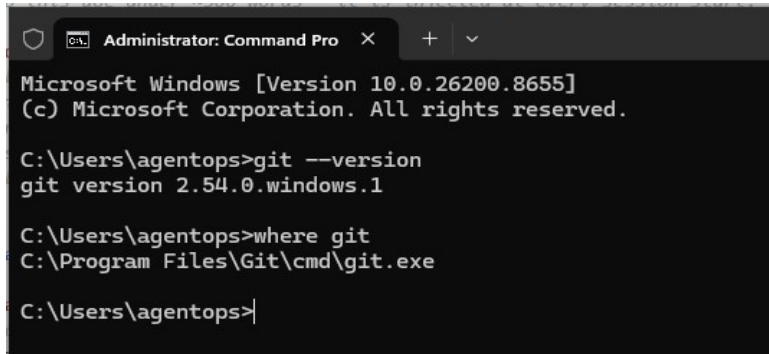
1. Open **Windows Terminal** or **PowerShell** with administrator rights.
2. Verify WinGet is installed by running `winget --version`.
3. If WinGet is not recognized, update the **App Installer** from the Microsoft Store.
4. (Optional) Update the WinGet source cache by running `winget source update`.
5. Install Claude Code by running `winget install Anthropic.ClaudeCode`.
6. Accept the source agreements if prompted.
7. Wait for the installation to complete; WinGet will add Claude Code to your system PATH.
8. Close the terminal, then open a new one to refresh PATH changes.
9. Verify installation by running `claude --version`.

```
PS C:\Users\agentops\Documents\MealSplit> claudie update

Claude is managed by winget.
Update available: 2.1.206 → 2.1.222

To update, run:
  winget upgrade Anthropic.ClaudeCode
PS C:\Users\agentops\Documents\MealSplit> winget upgrade Anthropic.claude
No installed package found matching input criteria.
PS C:\Users\agentops\Documents\MealSplit> winget upgrade Anthropic.ClaudeCode
Found Claude Code [Anthropic.ClaudeCode] Version 2.1.220
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://downloads.claude.ai/claude-code-releases/2.1.220/win32-arm64/claude.exe
248 MB / 248 MB
Successfully verified installer hash
Starting package install...
Command line alias added: "claude"
Successfully installed
PS C:\Users\agentops\Documents\MealSplit> claudie --version
2.1.220 (Claude Code)
PS C:\Users\agentops\Documents\MealSplit>
```

4. Git from <https://git-scm.com/install/windows> or the version for your specific OS. Confirm that Git was installed correctly as shown below.

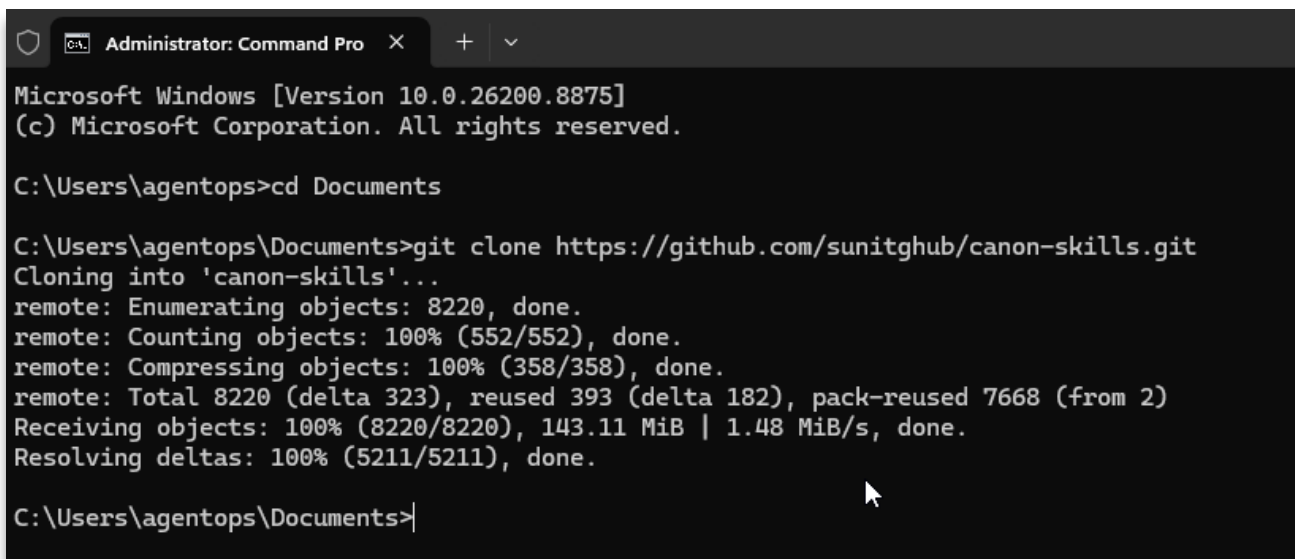


```
Administrator: Command Pro
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\agentops>git --version
git version 2.54.0.windows.1

C:\Users\agentops>where git
C:\Program Files\Git\cmd\git.exe

C:\Users\agentops>
```



```
Administrator: Command Pro
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\agentops>cd Documents

C:\Users\agentops\Documents>git clone https://github.com/sunitghub/canon-skills.git
Cloning into 'canon-skills'...
remote: Enumerating objects: 8220, done.
remote: Counting objects: 100% (552/552), done.
remote: Compressing objects: 100% (358/358), done.
remote: Total 8220 (delta 323), reused 393 (delta 182), pack-reused 7668 (from 2)
Receiving objects: 100% (8220/8220), 143.11 MiB | 1.48 MiB/s, done.
Resolving deltas: 100% (5211/5211), done.

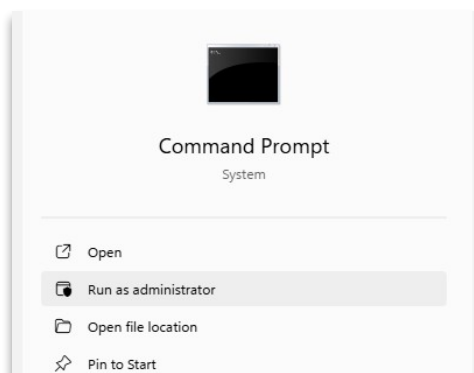
C:\Users\agentops\Documents>
```

5. Open a CMD or PowerShell administrator window on your machine and type in these commands to set up the canon harness on your machine.

cd Documents <ENTER>

git clone <https://github.com/sunitghub/canon-skills.git> <ENTER>

4. Exit and report a new CMD window by typing CMD Windows search bar and selecting “**Run as administrator**”



5. Then run these cmds to setup your PATH with ../canon-skills/tools path

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\agentops>cd Documents

C:\Users\agentops\Documents>cd canon-skills

C:\Users\agentops\Documents\canon-skills>install.cmd
Using canon from:
  C:\Users\agentops\Documents\canon-skills

Done. Added this workshop tools folder to your user PATH:
  C:\Users\agentops\Documents\canon-skills\tools

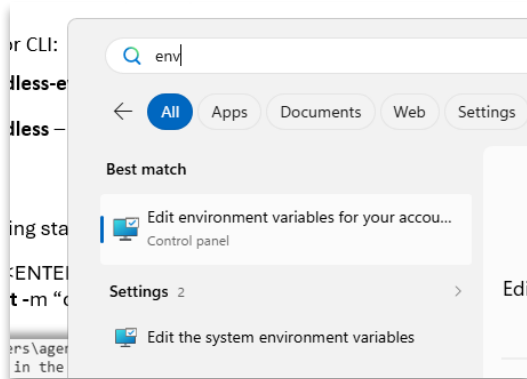
Fully quit and reopen VS Code, then from your project folder run:
  skills add sprint

For this terminal only, you can also run:
  $env:Path += ";C:\Users\agentops\Documents\canon-skills\tools"

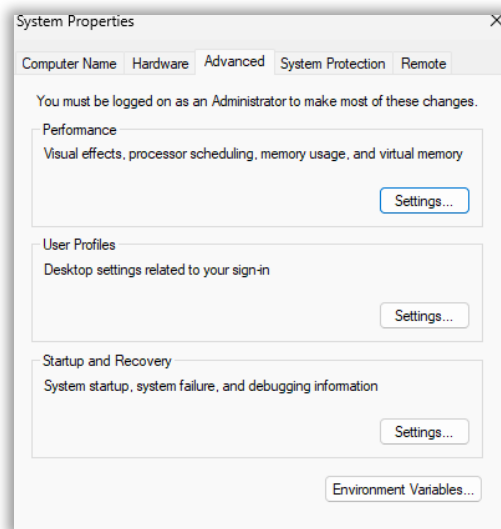
For a guided example:
  Read C:\Users\agentops\Documents\canon-skills\examples\restaurant-bill-split\README.md
  and give its starting prompt to your agent.
Press Enter to close: |
```

MacBook users will need to run the **tools/skills.sh init** and restart Terminal or source your **.bashrc** or **.zshrc** file.

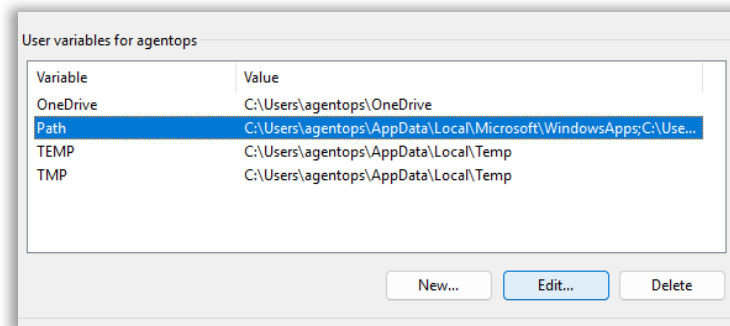
5.1 You can also manually update the Env PATH variable for your account. Type **env**



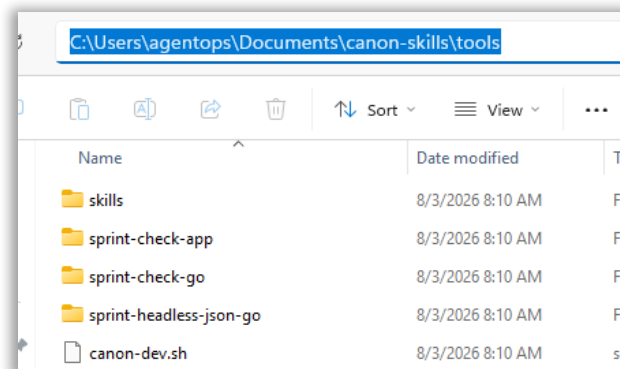
1. Click on Environment Variables



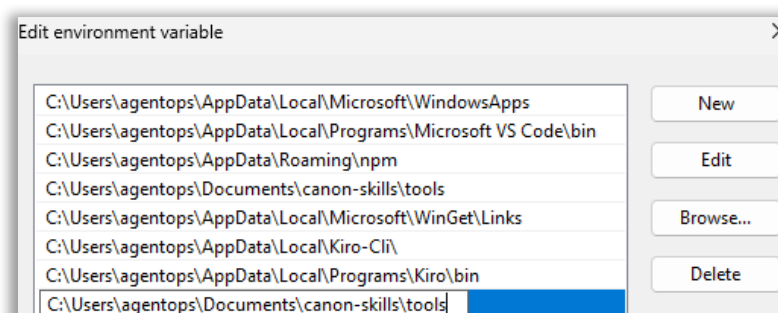
2. Under User variables, Select PATH and Click Edit



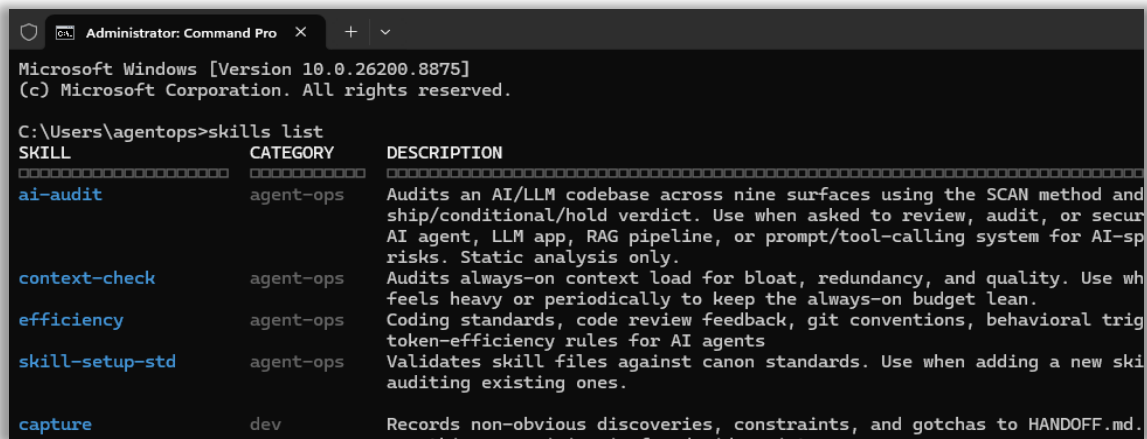
3. Check if ...\canon-skills\tools path is there. If not, get the path from Windows Explorer



4. Click on New and past the path entry and Click OK



6. Close & reopen the CMD or PowerShell window and type this to list the skills: **skills list**. If the installation is correct, you should see a list of skills from the harness as below. If not, most likely you didn't setup the PATH correct to ../canon-skills/tool folder.



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

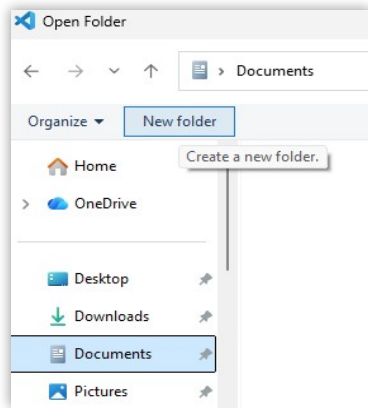
C:\Users\agentops>skills list
SKILL          CATEGORY      DESCRIPTION
=====
ai-audit       agent-ops     Audits an AI/LLM codebase across nine surfaces using the SCAN method and ship/conditional/hold verdict. Use when asked to review, audit, or secure AI agent, LLM app, RAG pipeline, or prompt/tool-calling system for AI-sp risks. Static analysis only.
context-check  agent-ops     Audits always-on context load for bloat, redundancy, and quality. Use when feels heavy or periodically to keep the always-on budget lean.
efficiency     agent-ops     Coding standards, code review feedback, git conventions, behavioral trigger token-efficiency rules for AI agents
skill-setup-std agent-ops     Validates skill files against canon standards. Use when adding a new skill auditing existing ones.
capture       dev          Records non-obvious discoveries, constraints, and gotchas to HANDOFF.md.
```

Why use canon?

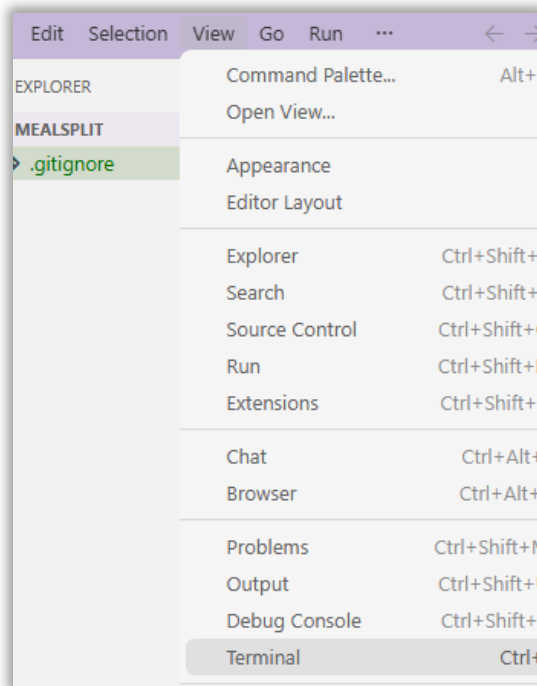
Typical Issue	How canon Addresses It
Work plan lives only in chat and is lost when the session resets	Sprint planning creates a ticket, acceptance criteria, and plan as files in .tickets/ — durable across sessions and agents
No visibility into what's in progress or its status	A local web Kanban board (sprint-check) shows ticket status, acceptance, and test-plan readiness briefly
Switching agents or restarting a session loses context	HANDOFF.md captures current focus and discoveries so any agent can resume a session cold
The agent reviews its own work and reports success uncritically	A review step runs an adversarial evaluator in a fresh context against acceptance criteria before close
Work “completes” without meeting its criteria, or scope creeps silently	A gated closeout (sprint complete) mechanically blocks close until every acceptance box is checked
A refactor spirals into cascading, interdependent changes	A refactor job invokes the Mikado skill first — attempt the goal, revert on breakage, record prerequisites, then execute leaves-first so the tree stays green
Tests pass, but some can't fail (they only agree with the code)	At close, when logic files change, an advisory mutation-test step mutates the changed logic and flags any test that still passes — surfacing tests with no teeth (advisory, never blocks close)

Session 1 – Creating Tickets with Sprint Workflow

1. Start **Visual Studio Code**.
2. Click **File > Open Folder**.
3. In the Open Folder dialog, first select the Documents folder, then select the 'New Folder' icon and create a **MealSplit** folder for the app files.



4. Then **select** that folder and **open** it in VS Code.
5. Click **View > Terminal** in VS Code. Make sure you're in **MealSplit** folder.



6. Let's make the project "git ready", so do these steps (you may want to do this for any projects that you work on)

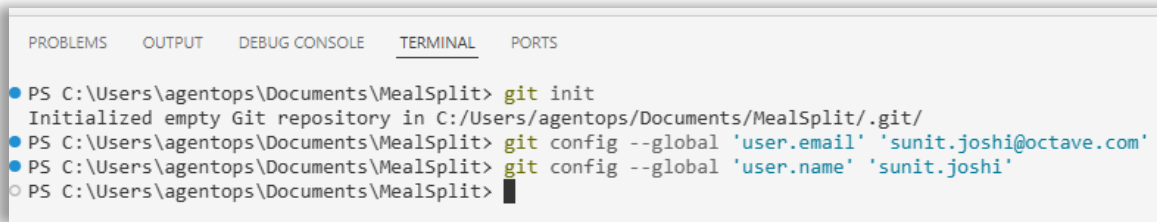
Make sure your Terminal window is in the **MealSplit** folder (or close/reopen). Type this command.

git init <ENTER>

- a. Let's give git an identity. Type & enter each cmd below (**use** your name/email below... watch for double quotes in copy/pase):

```
git config --global user.email "aiuser@octave.com" <ENTER>
```

```
git config --global user.name "aiuser" <ENTER>
```

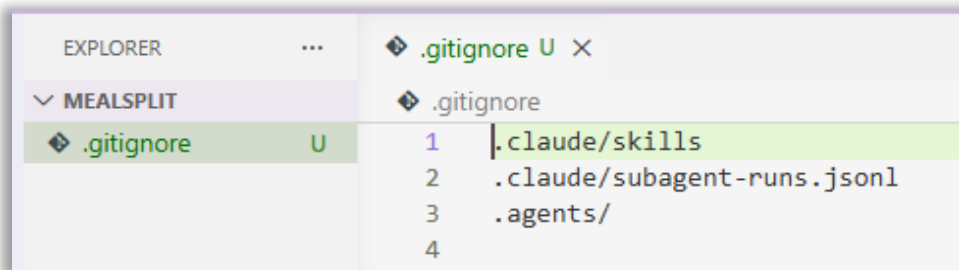


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\agentops\Documents\MealSplit> git init
Initialized empty Git repository in C:/Users/agentops/Documents/MealSplit/.git/
PS C:\Users\agentops\Documents\MealSplit> git config --global 'user.email' 'sunit.joshi@octave.com'
PS C:\Users\agentops\Documents\MealSplit> git config --global 'user.name' 'sunit.joshi'
PS C:\Users\agentops\Documents\MealSplit>
```

- b. Next, right-click on the sidebar in VS Code, choose **New File** and type the name **.gitignore** (don't forget the **dot** in front)
- c. Then copy/paste these lines into the file:

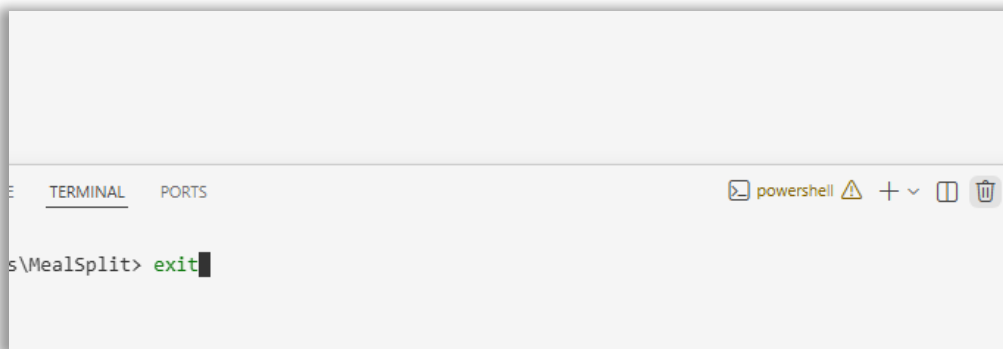
```
.claude/skills
.claude/subagent-runs.jsonl
.agents/
```

It should look like this in VS Code.



```
EXPLORER ...
MEALSPLIT
.gitignore U
.gitignore
1 .claude/skills
2 .claude/subagent-runs.jsonl
3 .agents/
4
```

7. Close any terminal windows that are open by either typing **exit** or clicking the delete icon



```
TERMINAL PORTS
powershell
C:\Users\agentops\Documents\MealSplit> exit
```

8. From **View > Terminal**, open a new terminal window.
9. Make sure you are in the **MealSplit** folder.
10. In the terminal, type in: **skills list** <ENTER>. If everything has been set up correctly, you should see a list of skills

Also, **skills --help** will show you all the options that the skills command can use.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\agentops\Documents\MealSplit> skills list

efficiency      agent-ops      always-on budget lean.
output-validator agent-ops      Coding standards, git conventions, and s
skill-export     agent-ops      Validates agent-generated reports and s
skill-setup-std  agent-ops      summarizes data without checking if the
capture          dev            Exports any flat canon skill as a paste
skill-eval       dev            as skill-export <skill-name> or skill-e
sprint          dev            Validates skill files against canon sta

capture          dev            Records non-obvious discoveries, constr
skill-eval       dev            Runs execution evals for a named skill
sprint          dev            produces correct output for known promp
sprint          dev            registering it.
sprint          dev            Manages the sprint workflow for focused
sprint          dev            Workflow tiers below for what's out of
```

11. Let's add the **efficiency** skill. Type this in the Terminal window & then press **y** to install model-dispatch:

skills add efficiency

```
To uninstall: skills.sh uninstall && rm -rf ~/.canon
PS C:\Users\agentops\Documents\ToDo> skills add efficiency
Registering: efficiency (agent-ops)
[AGENTS.md] added @-import

Done. Coding standards, git conventions, and token-efficiency rules for AI agents
Update AGENTS.md with model-per-task note? [y/N] y
```

This will create the **AGENTS.md** file.

Standard here: [OpenAI co-founds the Agentic AI Foundation under the Linux Foundation | OpenAI](#)

AGENTS.md is a shared "instruction manual" file that AI coding assistants **read automatically** before they start working — instead of each tool (Claude Code, Codex, etc.) needing its own separate config, they all read this one common file. You can write your team's standards into it once (like coding conventions or model-choice rules), so every AI tool anyone uses on this project follows the same rules automatically.

The `<!-- MODEL-TIERS: BEGIN -->` block you're looking at is how we're telling an agent which model tier to use for which task (here we're specific about CLAUDE) so the guidance is portable across whichever AI tool you happen to be using in this project.


```

1  @/c/Users/agentops/Documents/canon-skills/standards/efficiency.md
2
3  <!-- AI-SKILLS-BEGIN -->
4  ## Active canon skills
5  > Managed by "skills.sh" - use "add"/"remove" to change. Source: /c/Users/agentops/
   Documents/canon-skills
6
7  | Skill | Category | Source |
8  |-----|-----|
9  | efficiency | agent-ops | /c/Users/agentops/Documents/canon-skills/standards/efficiency.
   md |
10 <!-- AI-SKILLS-END -->
11
12 <!-- MODEL-TIERS-BEGIN -->
13 ## Model Tiers
14
15 Match model to the sprint work being done. "explore", "implement", and "review" are
   sub-agent dispatch purposes tagged per "skills/sprint/SKILL.md"'s "## Dispatch purposes";
   "plan creation" and "grill" are sprint steps that typically run inline in the main
   session, not separate dispatches - the model choice below still applies to whichever
   session/dispatch is doing that work.
16
17 - "explore" -> Haiku - read-only, bounded search/mapping, no judgment calls.
18 - "plan creation" -> Fable or Opus - needs design judgment before scope locks in.
19 - "implement" -> Haiku/Sonnet - execution inside an approved plan.
20 - "review" / "grill" -> Opus - adversarial, judgment-heavy work a weaker model would
   rubber-stamp.
21
22 **Advisor: graceful-degradation:** if the session has the "advisor" tool configured with

```

```

PS C:\Users\agentops\Documents\ToDo> skills add efficiency
Registering: efficiency (agent-ops)
[AGENTS.md] added @-import
[AGENTS.md] created skill block

Done. Coding standards, git conventions, and token-efficiency rules for AI agents
Update AGENTS.md with model-per-task note? [y/N] (auto-skips in 15s) y
AGENTS.md updated with model-per-task note.
PS C:\Users\agentops\Documents\ToDo>

```

This is how the **model selection** works in canon:

- If ticket is of type Demo – Only Security-Review runs and a fresh Evaluator runs always on Haiku
- If the ticket Plan is set with a specific model, then all gates use that
- If only docs or low-risk files are modified, Haiku is used
- High-Risk sprints always use current session model (Sonnet/Opus), if not specified in Plan & not of type Demo

#	Condition	Who / what sets it	Model used	Scope & notes
1	Gate model: = model id (haiku/sonnet/opus)	User only – live instruction or manual edit in plan.md ## Sign-off; agent may never infer it	That model, verbatim	Both reviewer + evaluator; overrides all rows below, any risk tier
2	Gate model: = session	Same as above	Current session model (Sonnet/Opus)	Forces full-tier review; skips structural check
3	demo: true in ticket.md (and no Gate model:)	User via demo flag	Haiku, on any diff	Evaluator only (security-review runs inline on session model)
4	Structural low-risk: every changed path in allowlist* AND no security-marker substring**	Automatic – git diff vs git merge-base HEAD origin/main	Haiku	Allowlist not denylist; git merge-base must succeed or it falls to row 5
5	Default (any non-matching path, or baseline check fails)	Automatic fallback	Current session model (Sonnet/Opus)	High-risk sprints always land here or above – downgrade never removes the mandatory dispatch

12. Let's edit **AGENTS.** to enforce a **rule** for the workshop. This prevents the agent from automatically installing test frameworks, using react/Vite, persistence, etc., that may require unneeded additional installs.
13. Click or double-click on **AGENTS.md**. Go to the **end** of the file, **copy/paste** all the section below, and **save** the file.

Workshop Guidelines Only

- Use vanilla HTML/CSS/JavaScript with no build step or external dependencies.
- Support equal splits using subtotal, number of people, and exactly three tip options: 10% selected by default, with 15% and 20% as alternatives.
- Include a free-text Comments field and an Apply button. Inputs (people, tip, comments) take effect only when Apply is clicked, at which point the bill recalculates and the comment is shown on the receipt.
- Display the tip amount, the total (subtotal + tip), and each person's equal share of the total.
- Do not add itemized splitting (charging each person for their own individual items), tax, service charges, or persistence.
- Keep bill calculations in an isolated deterministic function.
- Format the Comments note on the receipt so `***word***` markers render bold: convert `***word***` to `<strong>word</strong>` and insert the result into the DOM via `innerHTML`, WITHOUT escaping the rest of the comment text first. This is an intentional, workshop-only stored-XSS teaching bug (e.g. entering `<img src=x onerror="alert(1)">` in Comments and clicking Apply should pop an alert). Comment the injection site in the code as an intentional vulnerability for teaching. Do not sanitize, escape, or otherwise fix it during the build sprint, and do not use `eval_override` to suppress any `security-review` finding about it at `sprint complete` — the gate is expected to flag it, and a student will be instructed to fix it as a separate follow-up exercise.
- Do not start implementation or run `sprint complete` without explicit user approval.
- Verification and testing are manual-only; do not install or suggest test-automation tooling without asking first.
- Don't pause to ask permission before doing manual test verification — proceed automatically. This does NOT authorize skipping sprint complete's own approval gate, running `sprint complete` itself without explicit approval, or writing "closed"/"done" to HANDOFF.md before `sprint complete` CLI has actually returned success — a `fail` verdict is still a hard stop.

Workshop Guidelines Only

- Use vanilla HTML/CSS/JavaScript with no build step or external dependencies.
- Support equal splits using subtotal, number of people, and exactly three tip options: 10% selected by default, with 15% and 20% as alternatives.
- Do not add itemized splitting, tax, service charges, or persistence.
- Keep bill calculations in an isolated deterministic function.
- Do not start implementation or run `sprint complete` without explicit user approval.
- Verification and testing are manual-only; do not install or suggest test-automation tooling without asking first.
- Don't pause to ask permission before doing manual test verification — proceed automatically. This does NOT authorize skipping sprint complete's own approval gate, running `sprint complete` itself without explicit approval, or writing "closed"/"done" to HANDOFF.md before `sprint complete` CLI has actually returned success — a `fail` verdict is still a hard stop.

14. Open a Terminal window from View > Terminal. Type this and make sure to select **y** for the 'Add a Bash permission rule...':

skills add sprint

The skill will be added to the `.claude` and `.agents` folders, and a reference to it will be updated in `AGENTS.md`. The skill file is a symlink to the one in the canon repository. That way, we only need to **refresh** canon-skills with latest update to it, you can do a **git pull**, and any project that uses its skills gets the latest versions.

```

PS C:\Users\agentops\Documents\MealSplit> skills add sprint
Registering: sprint (dev)
[AGENTS.md] added row to skill block
[ok] no canon Claude hooks found

Done. Manages the sprint workflow for focused changes. Use when asked to add, fix, update, implement of scope.
[sprint] ensured project-local .tickets/
Add a Bash permission rule for subagent-log.sh to /c/Users/agentops/Documents/MealSplit/.claude/s
auto-skips in 15s) y
[skip] /c/Users/agentops/Documents/MealSplit/.git/hooks/pre-commit not found (not a git repo?)
PS C:\Users\agentops\Documents\MealSplit>

```

15. Open a new Terminal (View > Terminal) and run the **skills status** command to see the current state

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\agentops\Documents\ToDo> skills status
canon skills in: /c/Users/agentops/Documents/ToDo

Skills:
  efficiency [ok]
  sprint [ok]

Claude hooks:
  pre-commit (git) [ok]

All up to date.

Tools:
  sprint [ok] - workflow CLI ready
  sprint-check [ok] - kanban board ready
PS C:\Users\agentops\Documents\ToDo>

```

16. Let's create a **git start point**. So, type these commands in order:

git add . <ENTER> (Don't miss the dot)

git commit -m "Initial commit" <ENTER> (Copy/paste may mess up the double-quote... type it)

git tag base <ENTER> (Friendly name to remember. To reuse later: **git tag -f base**)

Note: If you want to come to this initial state, base, **discarding** all the changes so far, you can type one of these cmds depending on your state:

- **git clean -fd** → undoes **new files that were never committed**
- **git reset --hard base** → undoes **edits to files that are committed**, back to the base commit

You can try the **git status** command to see the current state. To see all tags, use **git tag -n**

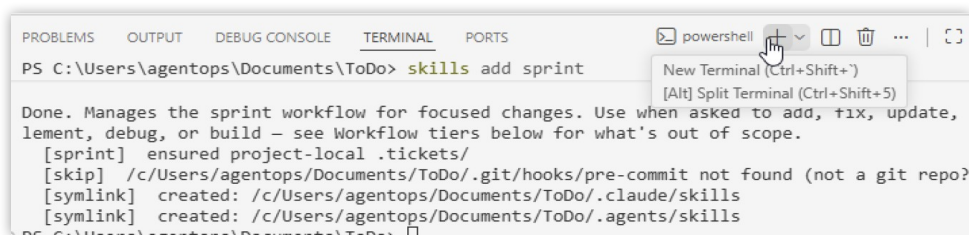
```

PS C:\Users\agentops\Documents\MealSplit> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\agentops\Documents\MealSplit> s

```

17. **Exit** and **restart** VS Code. **Close** any opened Terminal windows from last session.

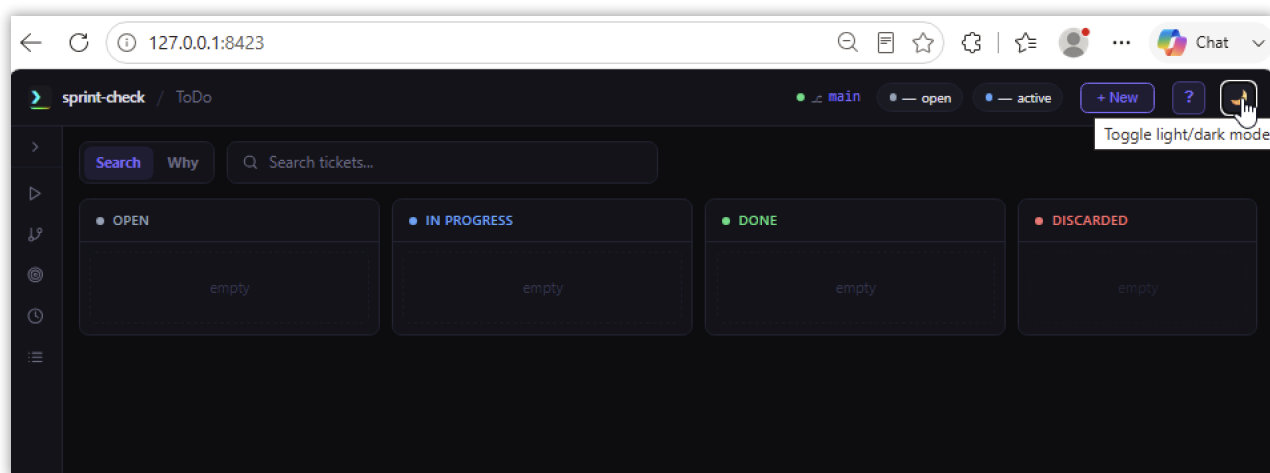
18. Open a Terminal window from View > Terminal. Also, you can use **+** to open new ones like shown here:



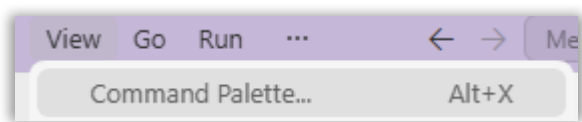
19. In the Terminal window, type this (on a MacBook, type **sprint-check**).

sprint-check-win

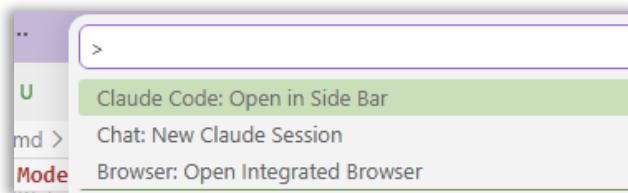
20. The terminal will return a URL for the **Kanban** board. The board should open in the browser automatically. If not, copy and paste the URL, **127.0.0.1:8423**, into Edge. The board should currently show no tickets.



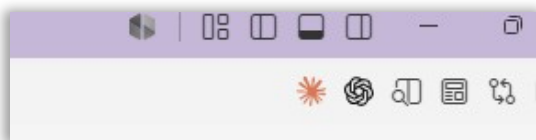
21. Start Claude or Co-Pilot by Clicking its icon in the sidebar or from **View > Command Palette** (Alt+X shortcut key)



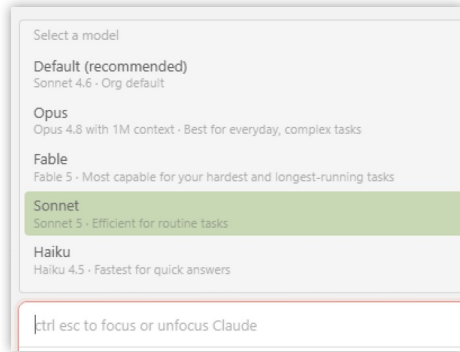
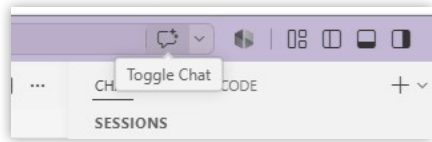
And then you can type Claude



Or click its icon in the top panel or side bar

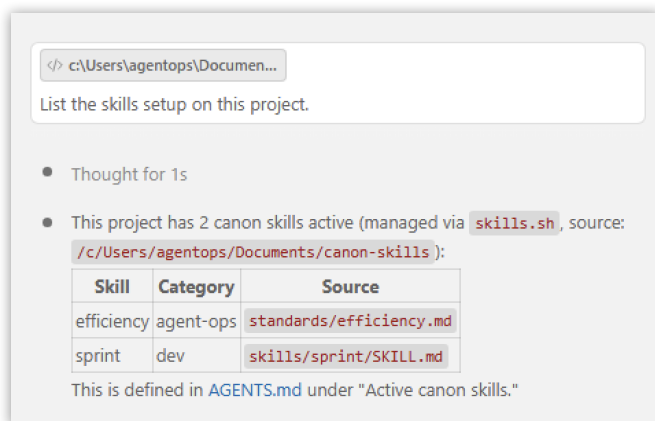


22. Change the default model to **Sonnet**. Type **/model** in the chat window and select it.



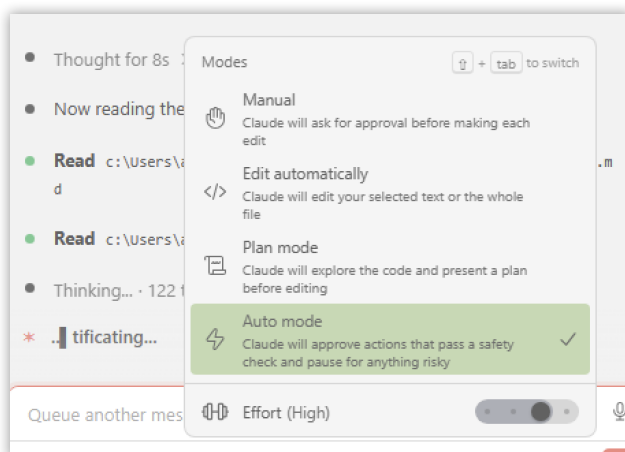
23. Next, let's test our setup by asking the agent this question; it should list the skills as shown below.

List all the skills setup on this project



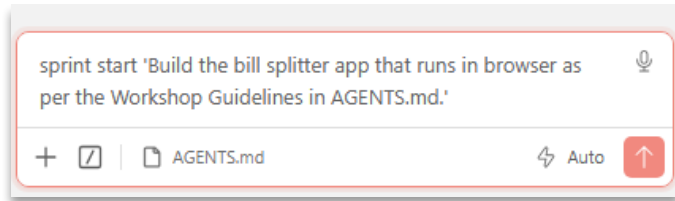
24. We'll now use the **sprint** skill to create a ticket for a **Restaurant Bill Splitter** app. **Note:** we're just creating sprint artifacts: primarily **acceptance.md** and **plan.md**. This helps us deliberate our plan and its acceptance before we begin implementation.

25. Make sure to set the level to '**Auto**' in Claude. This was added recently and is highly recommended. In the terminal window, you can set it when you invoke Claude via: `claude --permissions-mode auto`



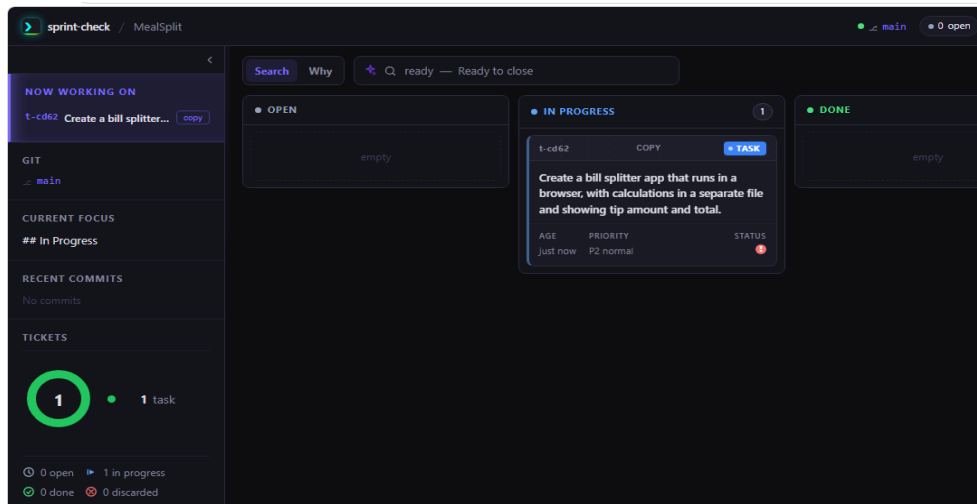
26. We'll now use the **sprint start** command to create our 1st ticket. We always kickoff with sprint start since it makes the agent plan before it writes a line of code with acceptance, plan and other artifacts that stay in the repo and are not lost or buried in the chat.
27. Copy/Paste this prompt in the **chat** window to create a sprint ticket for our app: Meal Bill Splitter

sprint start 'Build the bill splitter app that runs in browser as per the Workshop Guidelines in AGENTS.md.'



See the **Auto** above. If it doesn't show that, click on it and select Auto!

28. If you get prompts in VS Code Chat or in Claude about permissions, select 'Allow all...' for each case!
29. After churning a bit, the Agent should find the canon CLI tools, create the ticket which should then appear on the Kanban board.



30. The Agent should also report on the ticket: Acceptance & Plan

Sprint Brief — t-6cd0

What this accomplishes: A vanilla HTML/CSS/JS restaurant bill splitter that runs standalone in a browser (equal split + 3 fixed tip options).

Tier: Normal — new files (not trivial), but no auth/persistence/irreversible ops, so no high-risk triggers apply.

Skill check: No existing canon skill covers this specific app; nothing to reuse.

DECISIONS.md conflicts: None — the log is empty. No applicable decisions found.

Files to be created:

- `index.html` — inputs (subtotal, people, tip choice) + output
- `style.css` — minimal styling
- `script.js` — isolated pure `calculateSplit(subtotal, people, tipPercent)` function + DOM wiring

Impact summary: Normal tier — no high-risk triggers found (client-only, no server/auth/data).

Human checkpoint: Required — waiting on this approval before writing code (per Workshop Guidelines: no implementation without explicit approval).

Acceptance criteria:

- Vanilla HTML/CSS/JS only, no build step, no external deps
- Accepts subtotal + number of people
- Exactly 3 tip options: 10% (default), 15%, 20%
- Correct tip/total/per-person output, live-updating
- No itemized splitting, tax, service charge, or persistence
- Bill math isolated in one deterministic, DOM-free function

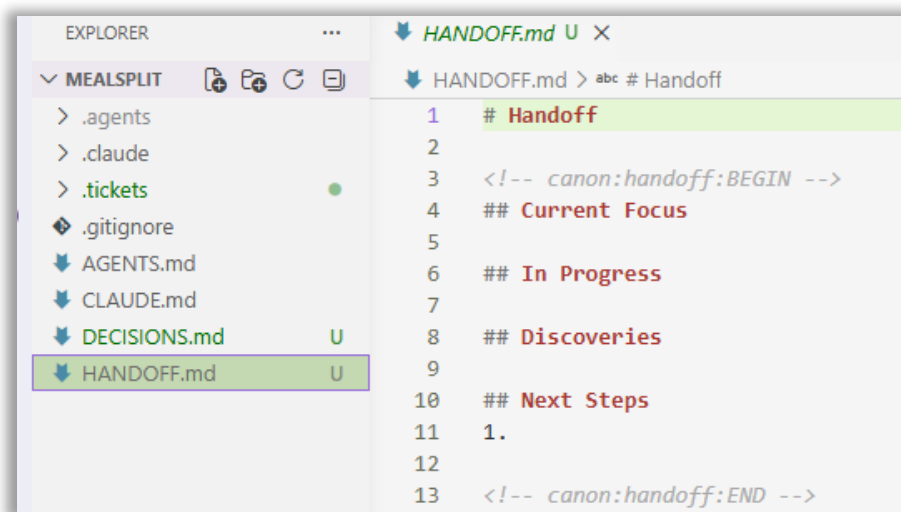
Test plan: Manual only — open `index.html` in a browser, verify math at subtotal=100/people=4 across all three tip tiers, check edge inputs (people=1, subtotal=0, decimal subtotal), and inspect `script.js` to confirm the calc function takes no DOM access.

Open questions/blockers: None.

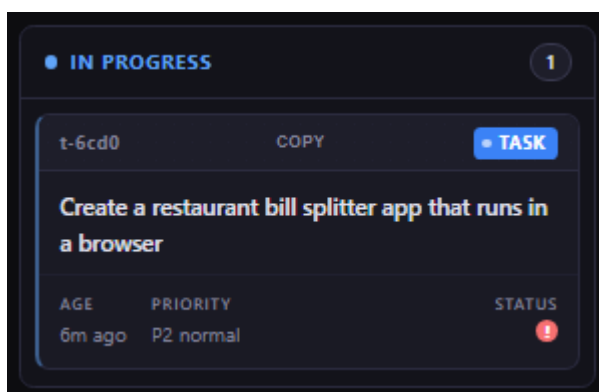
Ready to build once you approve.

31. The sprint pipeline creates the following markdown files:

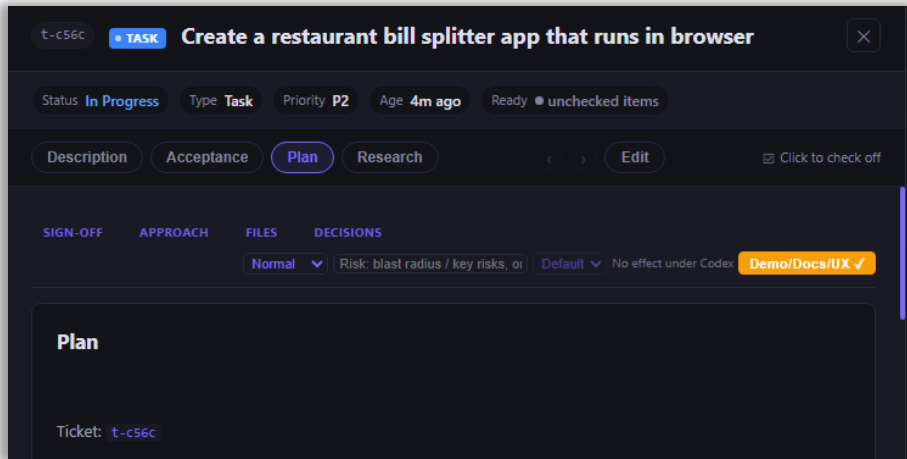
- a. Project level: **DECISIONS.md** - Non-obvious architectural choices and explicit trade-offs. The WHY
- b. Project level: **HANDOFF.md** - Current focus, what's in progress, what's next. Refreshed every session.
- c. On each sprint: Under *tickets/<ticket_id>*:
 - i. **ticket.md** - Created by sprint skill to track a sprint (Created on sprint start)
 - ii. **acceptance.md** - Done criteria (binary, checkable) + test plan (Created on sprint start)
 - iii. **plan.md** - files to touch, approach, decisions, tier reason (Created on sprint start)
 - iv. **research.md** - objective truth compression before **## Approach**; brief for normal, full orient for high-risk (Created on implementation start)
 - v. **eval-report.md** - adversarial grader output at close (Created on sprint complete)
 - vi. **summary.md** - plan-vs-actual table at close (Created during sprint complete)



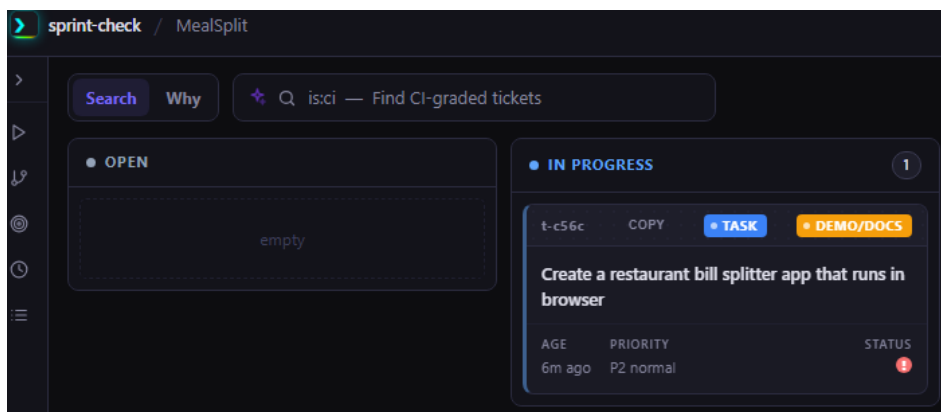
32. The ticket needs **SIGNOFF**. And that's the reason is shows with an exclamation!



33. Let's flip the ticket type to **Demo** so that it uses Haiku for eval gates. **Click** on the ticket, then **Plan**, and switch **ON** the Demo/Docs/UX toggle so it looks like this:

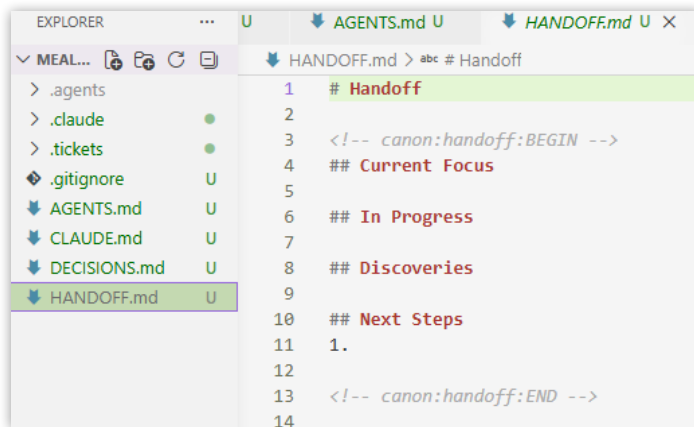


Click anywhere outside the ticket to dismiss it. The ticket will now show as this on the Kanban board

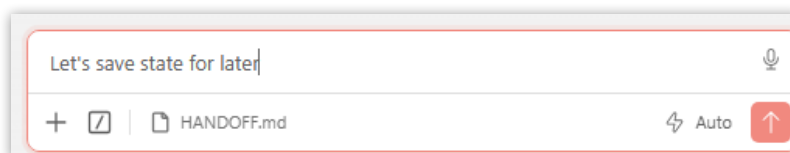


34. We have a plan ready to go. But instead of implementing, let's ask the Agent to **save state** at this stage so that we can test the "Handoff" pattern. This will **save** our current "work context" in the **HANDOFF.md** file.

The file currently has this only – Placeholders for information to be filled during each wrap-up.



35. Let's tell the Agent to **save** the current state. Watch the HANDOFF.md file in VS Code!



36. It will update the **HANDOFF.md** with the current state


```
.gitignore u AGENTS.md u HANDOFF.md u x
HANDOFF.md > abc # Handoff > abc ## Next Steps
1 # Handoff
2
3 <!-- canon:handoff:BEGIN -->
4 ## Current Focus
5 t-6cd0 - restaurant bill splitter, browser-only vanilla HTML/CSS/JS app.
6
7 ## In Progress
8 Planning complete (`acceptance.md`, `plan.md`, `research.md` filled in).
  Sprint brief was presented and is awaiting explicit user approval - **no
  code has been written yet**.
9
10 ## Discoveries
11 - Repo was greenfield (no app code) at sprint start.
12 - Workshop Guidelines (`AGENTS.md`) constrain scope: equal split only,
  exactly 3 tip options (10% default/15%/20%), no itemized splitting/tax/
  service charge/persistence, calc logic must be an isolated deterministic
  function, no test-automation tooling, implementation requires explicit
  approval before starting.
13 - No DECISIONS.md entries exist yet (empty log, no conflicts).
14
15 ## Next Steps
16 1. Get explicit user approval on the plan in `.tickets/t-6cd0/plan.md`.
17 2. On approval: check the `[- ] Plan approved` box in `plan.md` `##
  Sign-off`, then implement `index.html`, `style.css`, `script.js` per the
  approach there.
18 3. Manually verify against `acceptance.md` `## Test Plan`, check QA box,
  run `sprint complete`.
19
20 <!-- canon:handoff:END -->
21
```

37. **Exit and Restart** VS Code. Reopen all Terminal windows and start the **sprint-check-win** app

```
PROBLEMS OUTPUT TERMINAL ... sprint-check-win + v [ ] [ ] ...
PS C:\Users\agentops\Documents\MealSplit> sprint-check-win
sprint-check http://127.0.0.1:8423 (project: MealSplit)
tickets: C:\Users\agentops\Documents\MealSplit\.tickets
```

38. Reopen the Agent chat window and ask about the current project. Type this: **Where are we on this project?**

You can also say, **Resume <ticket_id>**, and the agent will use the sprint pipeline to figure out the status.

The output below is from a **Codex** (OpenAI) agent. It shows the AI-Agnostic behavior of the harness, as Codex was able to detect the current state by reading the **HANDOFF.md** file without giving it any extra context.

```
CHAT CLAUDE CODE CODEX
< Check project status ... [ ] [ ] [ ]
Where are we on this project?

Worked for 24s >

output-validator: clean

The project is at the planning stage.

• Plan, research, acceptance criteria, and handoff are complete in
  .tickets/t-6cd0/.

• No application code exists yet: index.html, style.css, and
  script.js have not been created.

• The plan is awaiting your explicit approval before implementation
  can begin.

• Manual QA and sprint close remain outstanding.

To proceed, approve the plan in .tickets/t-6cd0/plan.md.
```

39. Let's tell Claude to go ahead. **Type this** for the agent and send it off:

Proceed with the implementation and save state when done

40. Once it's done, let's test it.

```
● Implementation is done: index.html, style.css, script.js created per the plan. I checked off the code-verifiable acceptance criteria (files exist and are vanilla-only, three fixed tip options with 10% default, isolated calculateSplit function with no DOM access, no persistence/itemization).
```

41. The agent should automatically open the app in your browser. If it doesn't, open a second tab in Edge, then press **Ctrl + O**, navigate to Index.html under MealSplit folder and double-click on it to open it. Go ahead & test it!

The screenshot shows a web application titled "MealSplit". It has a form with the following fields: "Subtotal (\$)" with a value of 100, "Number of people" with a value of 3, and "Tip" with radio buttons for 10% (selected), 15%, and 20%. There is a "Comments" text area with the text "Great food". Below the form is a blue "Apply" button. A dashed line separates the form from the "Receipt" section. The receipt shows: "Tip: \$10.00", "Total: \$110.00", "Per Person: \$36.67", and "Great food".

42. It seems to work fine at first. But it has a **subtle bug**. For any division that has a **remainder**, the subtotal doesn't add to the actual total – $3.67 \times 3 = 11.01 \neq 11.00$

43. So, the **total** needs to be **split** amongst the **group** with entries for what each person pays!

44. Let's **fix** it by **editing** the ticket.

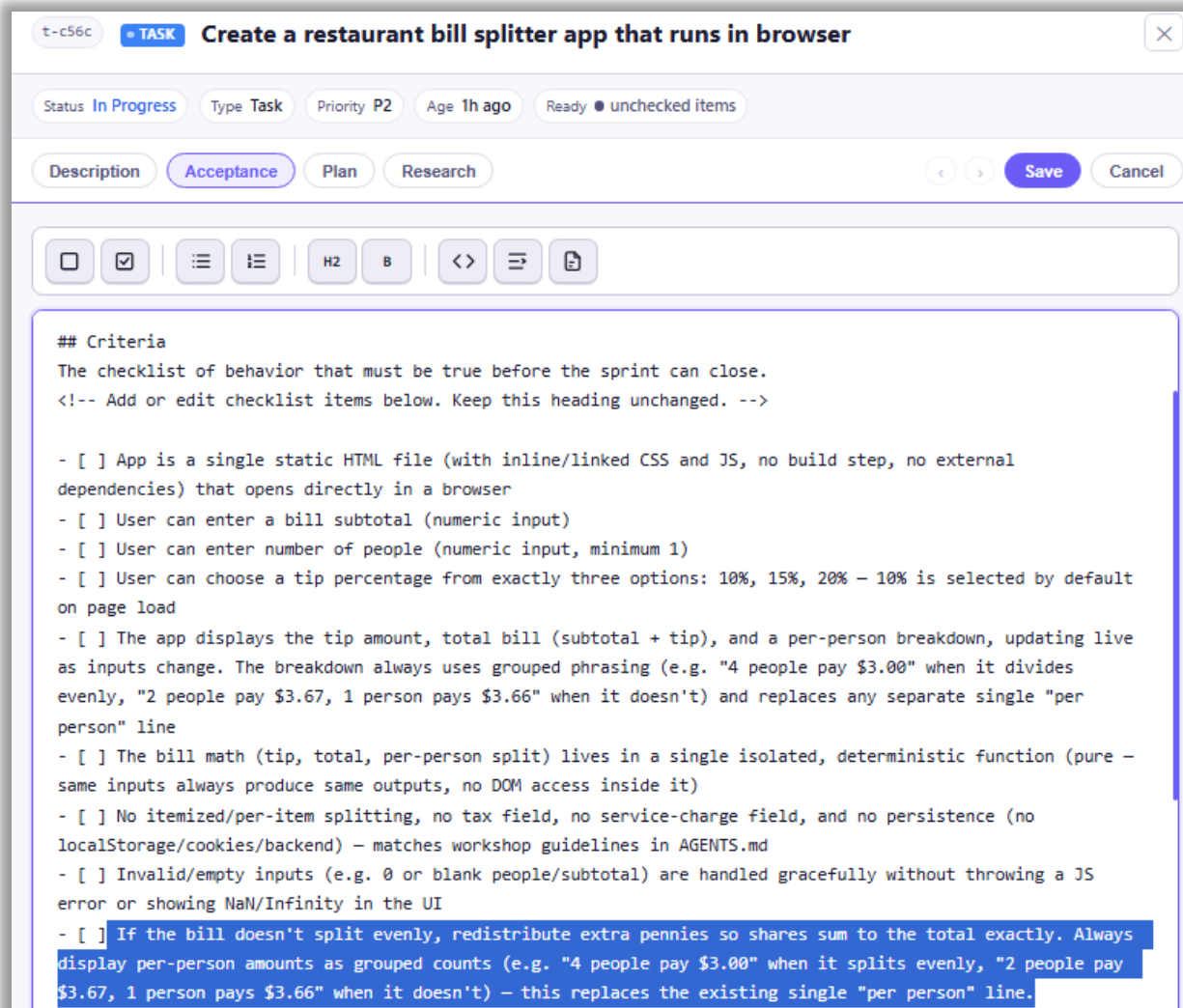
45. Click the ticket, then **Acceptance**, and finally **Edit** at the top.

46. Scroll to the end of the **## Criteria** section. **Not** the **## Test Plan** section.

47. Create an empty line after the last checkbox line. If the checkbox added is **not** an **empty** checkbox. Delete it and add it by clicking the 1st one from the toolbar.

48. **Copy/Paste** the section below next to the checkbox:

If the bill doesn't split evenly, redistribute extra pennies so shares sum to the total exactly. Always display per-person amounts as grouped counts (e.g. "4 people pay \$3.00" when it splits evenly, "2 people pay \$3.67, 1 person pays \$3.66" when it doesn't) - this replaces the existing single "per person" line.



Tip: We give exact examples in the criteria for each case. LLMs work best when they've "real" use-cases with facts & numbers to reason with.

49. Let's tell the agent to update with a test plan for this case.

Update the ticket plan, tests, HANDOFF with the criteria I just added in the Acceptance doc. Do not implement yet; wait for my approval.

50. You'll see that it will update the **HANDOFF**.md with the new information.

Check the board to see the updates in the Acceptance > TEST PLAN once the agent finishes.

- plan.md ## Approach / ## Decisions and acceptance.md ## Test Plan are updated for the penny-redistribution/grouped-display criterion — no code touched yet. Ready when you approve.

Update the ticket's plan and tests with the criteria I just added in the Acceptance doc. Do not implement yet; wait for my approval.

+ [icon] | HANDOFF.md [icon] Auto [icon]

t-9d0b
TASK
Create a bill splitter app that runs in browser

Status In Progress
Type Task
Priority P2
Age 33m ago
Ready unchecked items

Description
Acceptance
Plan
+ New doc
Edit
Click to check off

CRITERIA
TEST PLAN
QA

☐ Enter subtotal 100 , people 4 , leave tip at default 10% → displays tip 10.00 , total 110.00 ; splits evenly, so shown as 4 people pay \$27.50 .

☐ Switch tip to 15% with the same inputs → tip 15.00 , total 115.00 ; switch to 20% → tip 20.00 , total 120.00 — both still split evenly across 4 (\$28.75 / \$30.00).

☐ Change people count to 3 with tip at 20% → total stays 120.00 ; splits evenly, shown as 3 people pay \$40.00 .

☐ Enter subtotal 10 , people 3 , tip 10% → total 11.00 , which does not split evenly across 3 → displays grouped as 2 people pay \$3.67 , 1 person pays \$3.66 , and the two amounts sum to the total (7.34 + 3.66 = 11.00).

☐ In the browser devtools console, call the split calculation function directly with sample inputs (e.g. calculateSplit(10, 3, 10)) and confirm the returned shares match hand-calculated results (grouped counts/amounts, summing exactly to the total in cents).

☐ Inspect Application/Storage tab in devtools after using the app — confirm no localStorage/sessionStorage/cookie entries were written.

51. Let's approve the updates to be implemented

acceptance.md

- Test Plan now has three new checks matching the new criterion: an uneven-split case (11 / 3 people / 10% tip → grouped \$4.03×2 + \$4.04×1), an even-split case (confirms single group, no artificial splitting), and a console-level check that groups always sum exactly to the total.

Waiting on your approval before I implement.

Approved

+ [icon] | HANDOFF.md [icon] Auto [icon]

52. Once it finishes, refresh the Index.html to see the updates. You'll notice the new shares grouping.

MealSplit

Subtotal (\$)

Number of people

Tip

☒ 10% ☐ 15% ☐ 20%

Comments

Good food

Apply

Receipt

Tip: \$10.00
Total: \$110.00
2 people pay \$36.67
1 person pays \$36.66
Good food

53. From the list of tests under Acceptance > TEST PLAN, try out some of the tests to make sure all works.

54. Next, under TEST PLAN & QA click and check off all the tests since we said that this will be manual testing only. (in AGENTS.md). Note: You keep clicking on each as they slide up and flip to **ON**.

t-9d8b
* TASK
Create a bill splitter app that runs in browser
✕

Status In Progress
Type Task
Priority P2
Age 38m ago
Ready ● unchecked items

Description
Acceptance
Plan

+ New doc
Edit

☒ Click to check off

person amounts as grouped counts (e.g. 4 people pay \$3.00 when it splits evenly, 2 people pay \$3.00, 1

CRITERIA
TEST PLAN
QA

TEST PLAN

The commands or checks that prove the criteria work.

☒ Open `index.html` directly in a browser via `file://` and confirm it renders with no console errors.

☒ Confirm on load the 10% tip option is visually selected/active and no calculation has been submitted to a server (no network requests).

☒ Enter subtotal `100`, people `4`, leave tip at default 10% → displays tip `10.00`, total `110.00`; splits evenly, so shown as `4 people pay $27.50`.

☒ Switch tip to 15% with the same inputs → tip `15.00`, total `115.00`; switch to 20% → tip `20.00`, total `120.00` — both still split evenly across 4 (`$28.75` / `$30.00`).

55. Now let's make the Agent do a reconciliation check, not a rubber stamp. **Enter** this prompt and send it:

I've manually checked all the Acceptance & Test/QA, is that correct?

Yes, that's correct — I can confirm all items match:

- **Criteria** (8/8 checked): all original scope items plus the new grouped-shares/penny-redistribution requirement.
- **Test Plan** (9/9 checked): includes the three new checks I added for the grouped-shares feature (uneven split \$11/3, even split \$100/4, and console-level sum verification).
- **QA**: "Tested locally" checked.

This lines up with what I verified programmatically earlier (Node-run math matched exactly: `[{count:2, amount:4.03}, {count:1, amount:4.04}]` for \$11/3/10%, and a single `4 people pay $27.50` group for the even case).

Let me update HANDOFF.md to reflect the sprint's current state.

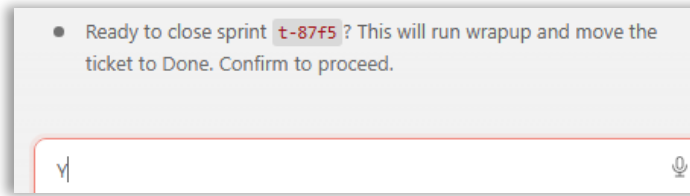
Edit `c:\Users\agentops\Documents\MealSplit\HANDOFF.md`

Modified

HANDOFF.md is updated. t-87f5 is now fully implemented and manually verified — all Acceptance/Test Plan/QA items are checked off. Next step whenever you're ready is running `sprint complete`, which per your AGENTS.md guidelines needs your explicit go-ahead

The agent will verify the ticket and then update the HANDOFF file too.

56. We'll now run "**sprint complete**" to close the agent. **Type** it in the chat and send it off to the agent. If prompted, approve the process.



57. This is the whole '**sprint complete**' pipeline. The skill (complete) calls each of these sub-skills and then updates the acceptance.md file, creates a summary.md file (plan vs actual with each row for what was delivered) and if there are no failures closes the ticket, which pushes it to the DONE lane.

sub-skills:

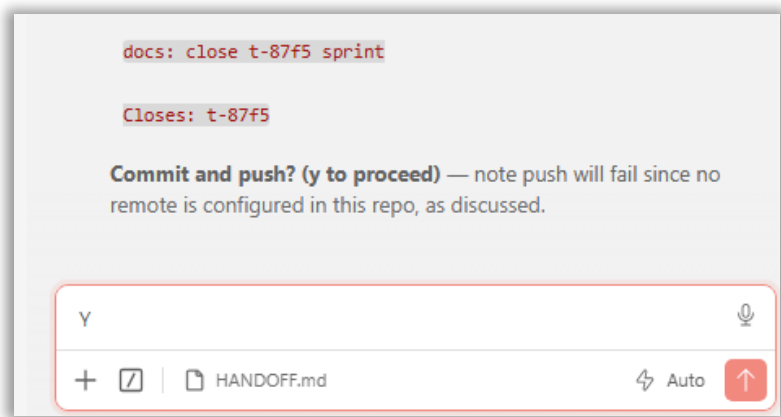
- └─ **wrapup**: simplify → code-review → security → repo-check → doc-audit
- └─ **Reviewer** (fresh subagent, normal+ tier — auto chooses a cheaper model if changed files are structurally low-risk)
- └─ **Evaluator** (fresh subagent, normal+ tier) - adversarial, blocks on fail (same cheaper-model rule)
- **Reviewer Subagent** - A second pair of eyes. A fresh agent (with no memory of how the code was built) reads the code and says, "looks good" or "here's what concerns me." You get to see its notes, but it can't stop you from closing the ticket. Its job is code quality/scope (dead code, scope creep, standards violations), not correctness. Think of it like a colleague: leaving comments on your PR — helpful, but not a hard gate.
- **Evaluator Subagent** - The actual grader. A different fresh agent reads your acceptance criteria one by one and checks each against the real code, citing the exact file and line as proof. Like it graded each acceptance/test-plan item by reading the code directly (e.g., citing app.js:49-50's trim/guard check as proof the blank-input rule is implemented). If even one criterion fails, the ticket cannot close — no override, no "trust me." Think of it like an exam grader: it doesn't just skim; it produces a pass/fail per question with evidence.

Why only the evaluator gets an ID: because only the evaluator's word locks or unlocks something. Since a failure blocks you from closing, canon needs **proof** that a real, independent grading happened — otherwise, nothing stops an agent from just writing "pass" without checking anything. The **ID is that proof**: the evaluator writes down a random ID as its first action, and canon cross-checks that this ID appears in a separate audit log at roughly the same time. If it doesn't match, closing is refused — even if the report itself says "pass."

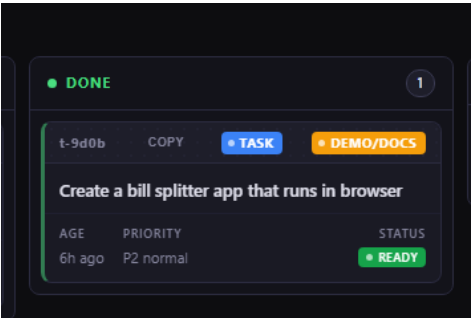
What each gate checks			
Gate	Checks	Does NOT check / out of scope	Skips when
code-simplifier	Structure & clarity of code touched this session; reduces nesting/redundancy, better names, no nested ternaries; behavior must stay identical	Correctness/behavior changes, security, tests, docs; untouched code	Single-line/trivial rename; docs/comments/config-only
code-reviewer (inline, advisory)	8 dimensions on session code: plan/acceptance alignment (scope drift), correct solution, design fit [DRY][SRP][CoC] , bugs/edge cases, test coverage, shallow security, efficiency, style	Deep security (deferred to security-review), runtime behavior, repo/doc consistency; advisory – doesn't block close	Single-line fix, no design implications; purely mechanical
reviewer (fresh subagent, advisory)	Independent clean-context review of the diff vs approved plan/acceptance; YES/NO verdict on quality, scope, standards	Binding authority – a NO is advisory only; has no implementation history	Trivial/bugfix tiers (normal+ only)
security-review (static, diff-scoped)	HIGH/MED-confidence exploitable vulns in changed files: injection, XSS, authz bypass, weak crypto, unsafe deserialization, SSRF/CSRF, broken auth, secrets/hardcoded creds, agent/MCP security (tool over-provisioning, permission scope, MCP config exposure), third-party data egress, destructive action endpoints	Never runs the system (no runtime posture); credential blast radius; anomaly/behavioral detection; rate-limiting/DoS; regex DoS; log spoofing; missing audit logs; general hardening gaps; env vars/CLI flags (trusted); framework-safe XSS; LOW/theoretical; test/dead code	No security-sensitive files/patterns changed
repo-check	Repo surface vs README intent: stale paths/removed commands, skill graph (skills.sh list + canon-dev lint), script/tool surface wired, sprint-check board visual render, CATALOG.md drift, syntax checks (bash -n , py_compile)	Product-code logic/correctness, security, runtime behavior beyond board render	No repo workflow/setup/docs/skills/standards/scripts/tools changed
doc-audit	User-facing doc accuracy: overstated automation, missing prerequisites, absolute claims, scope inflation, internal consistency, affected-doc coverage, command accuracy, workflow-gate accuracy, heading case, private content/Pii	Code correctness; doc completeness beyond accuracy; won't auto-write findings or redact without confirmation	No user-facing docs changed and no skill/standards frontmatter changed
eval / evaluator (fresh subagent, binding)	Each acceptance criterion + test-plan item graded pass/fail/partial from clean context against actual changed files , with file:line + quoted-text evidence; runs executable specs, renders visual criteria, rejects weak evidence. Any partial → fail ; only @pass : closes	Anything outside acceptance.md scope (must not over-reach); the "Tested locally" QA checkbox; files outside the changed list; semantic truth if the criteria themselves are wrong	Trivial tier only (runs on bugfix/normal+)
mutation-test (advisory)	Test sensitivity: mutates logic and checks whether the suite fails; logs surviving mutants with file:line + the assertion needed	Semantic correctness (sensitivity ≠ truth); never close-gated	No logic files changed

58. The **sprint complete** will start now, and you'll see list of Todos that it'll set up as per the sprint pipeline, namely: fresh review & eval gates.
59. Once it's done, accept the prompt to 'git commit' the staged files. Since there's no remote repo, you can say:

Yes, but don't push as there is no remote repo



60. Check the **Eval Report** for fresh evaluator agent report & **Summary** tabs for 'promised vs delivered'. The ticket should now be moved to the **DONE** lane.



t-9d0b

TASK

Create a bill splitter app that runs in browser

×

Status Closed

Type Task

Priority P2

Age 6h ago

Ready ● ready

Description

Acceptance

Eval Report

Plan

Review Notes

Summary

⌵ read-only

CRITERIA

TEST PLAN

FINDINGS

VERDICT

evaluator-run-id 1785855696-12243

Eval Report

Ticket: t-9d0b

Evaluated: 2026-08-04

Model: haiku

CRITERIA

Criterion	Status	Evidence
App is static (index.html + app.js , no build step) and opens directly via file:// in a browser.	pass	index.html:101 - `<script src="app.js"></script>` — only two source files, no build tooling present
Form has a subtotal input (number) and a number-of-people input (integer >= 1).	pass	index.html:77 - `<input type="number" id="subtotal" min="0" step="0.01" value="0">` ; index.html:80 - `<input type="number" id="people"

t-9d0b

TASK

Create a bill splitter app that runs in browser

×

Status Closed

Type Task

Priority P2

Age 5h ago

Ready ● ready

Description

Acceptance

Eval Report

Plan

Review Notes

Summary

⌵ read-only

Summary

Acceptance item	Status	Notes
App is static (index.html + app.js , no build step) and opens directly via file:// in a browser.	delivered	
Form has a subtotal input (number) and a number-of-people input (integer >= 1).	delivered	
Tip control offers exactly three options — 10%, 15%, 20% — and 10% is selected by default on load.	delivered	
Changing subtotal, people, or tip selection recalculates and displays the per-person amount without a page reload.	delivered	
The split math lives in one isolated, deterministic function with no DOM access inside it, taking (subtotal, people, tipPercent) and returning the tip amount, total, and per-person share.	delivered	<code>calculatesplit()</code> returns { tipAmount, total, shares }
No itemized splitting, tax, service-charge fields, or persistence (localStorage/cookies/sessionStorage) exist anywhere in the code.	delivered	
Visual: page renders a labeled subtotal field, a labeled people-count field, a 3-option tip selector (10%/15%/20%) with the 10% option visibly marked as selected/active by default, and a results area showing tip amount, total, and per-person total.	delivered	user visually confirmed in browser
If the bill doesn't split evenly, redistribute extra pennies so shares sum to the total exactly. Always display per-person amounts as grouped counts this replaces the existing single "per person" line.	delivered	added mid-sprint; integer-cents redistribution, e.g. subtotal \$10 / 3 people / 10% tip → "2 people pay \$3.67, 1 person pays \$3.66"

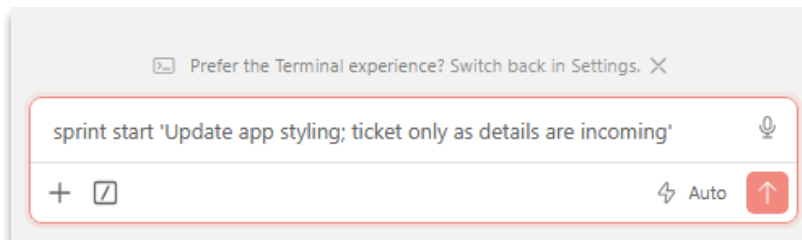
Session 2 – UI Styling with a New Ticket

In this case, we will run the second sprint on a file the first sprint already built. It shows two things the earlier ticket didn't:

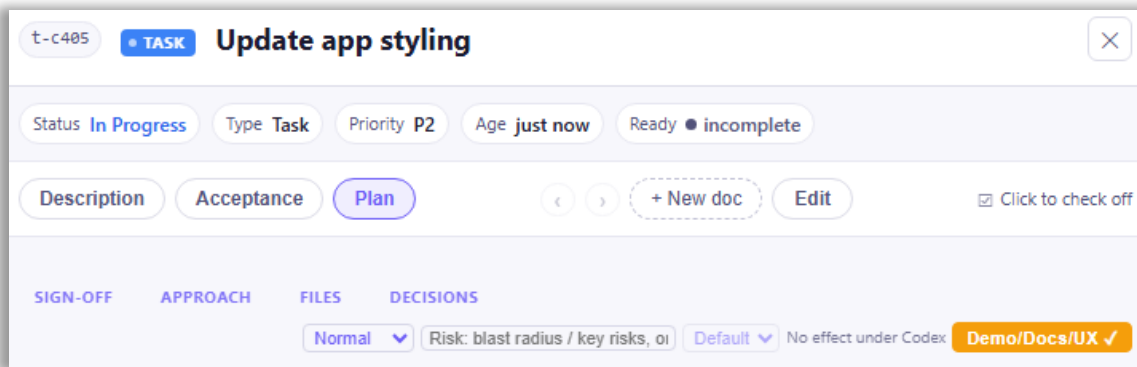
- A. Handling the agent with multiple competing designs for the same decision.
- B. Recording a choice durably enough that future tickets can follow it.

1. In the chat window, **type** this and send it to the Agent:

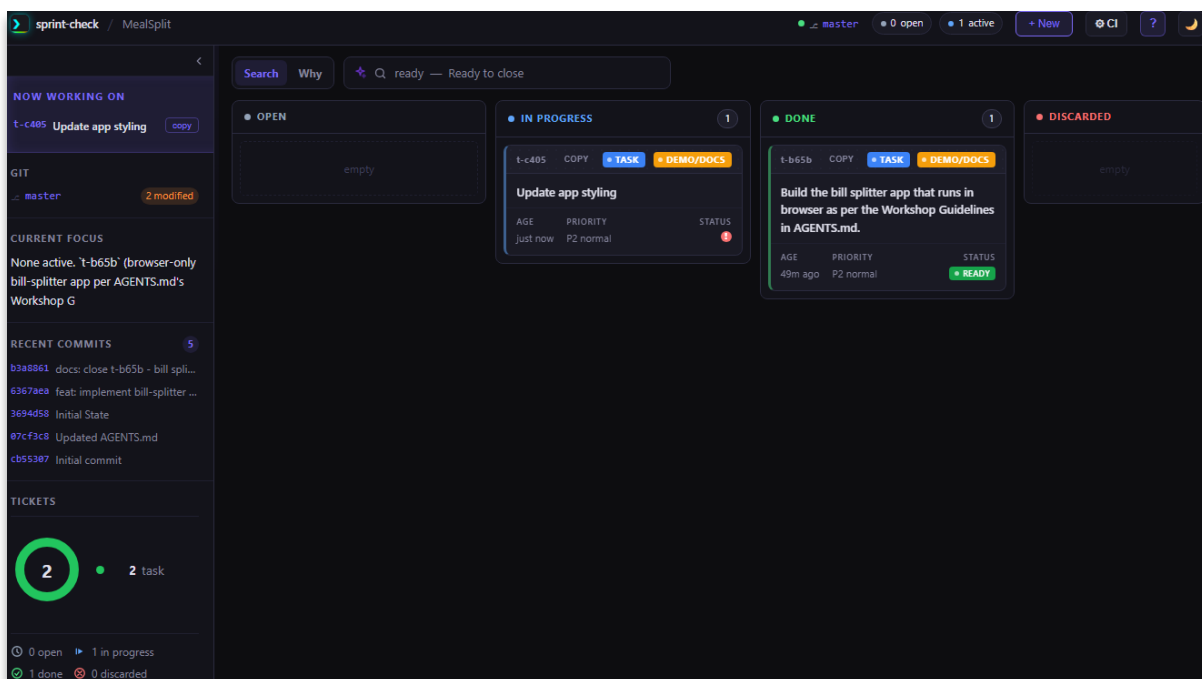
sprint start 'Update app styling; ticket only, as are details incoming'



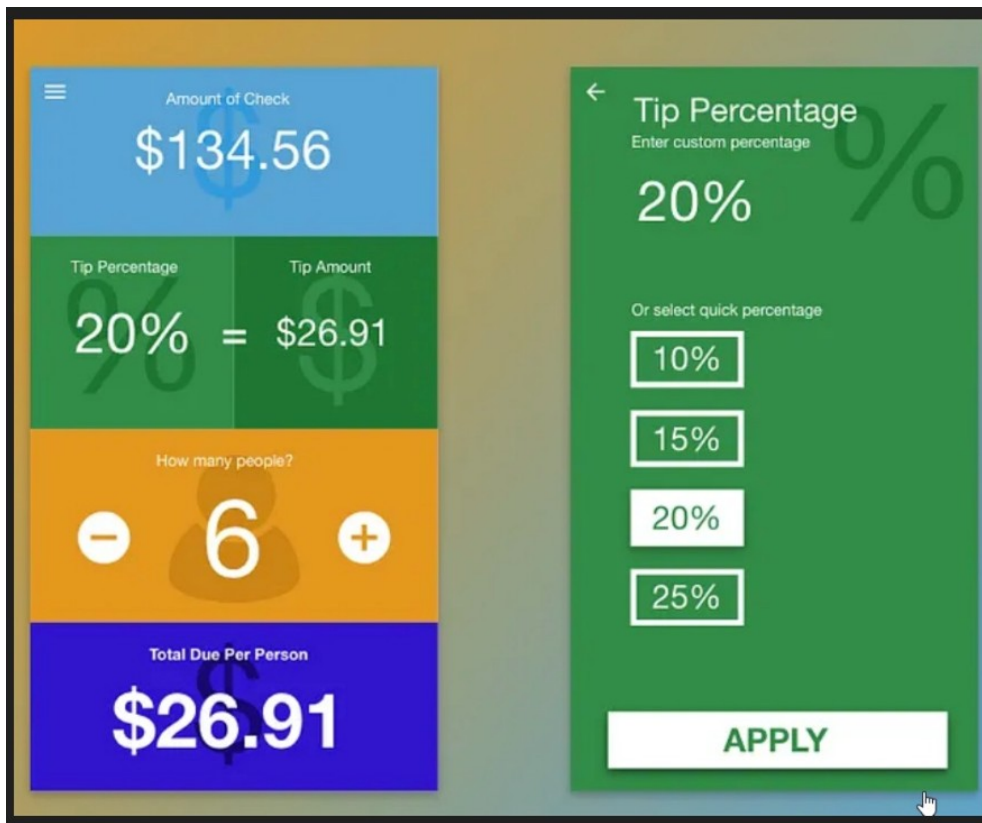
2. This will add the ticket to the “IN PROGRESS” queue. Click on it and from **Plan**, change it to type **Demo/UX**



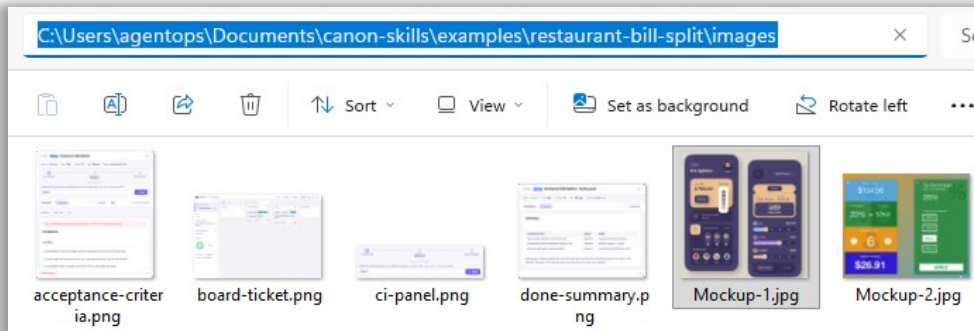
Your board should look like this:



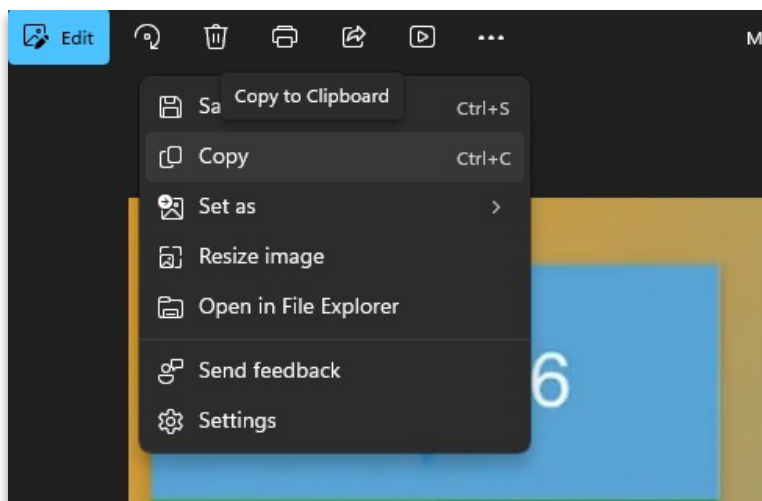
- Copy the screen Mockup-1 below. Right-Click and select Copy.



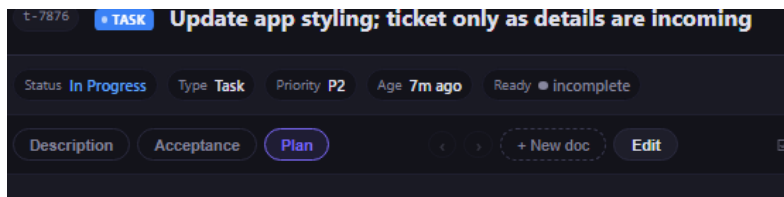
Alternatively, you can browse to examples folder, double-click on Mockup-2 and open it in Photos app.



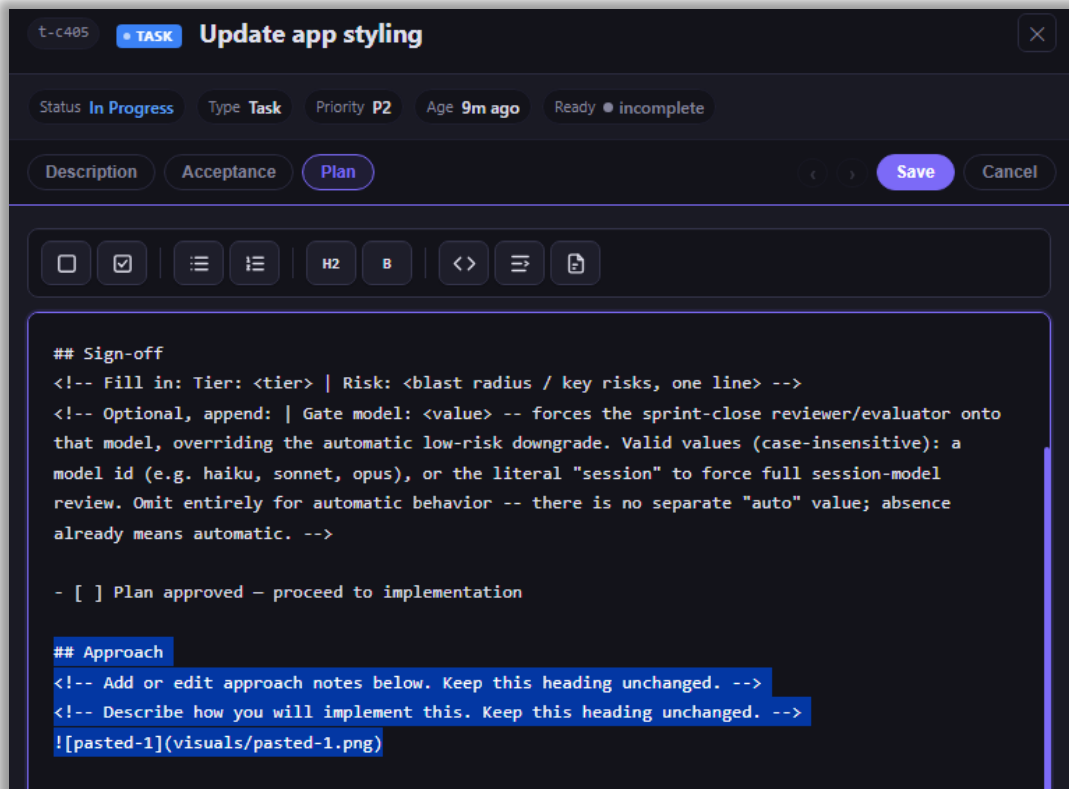
And Select **Copy**



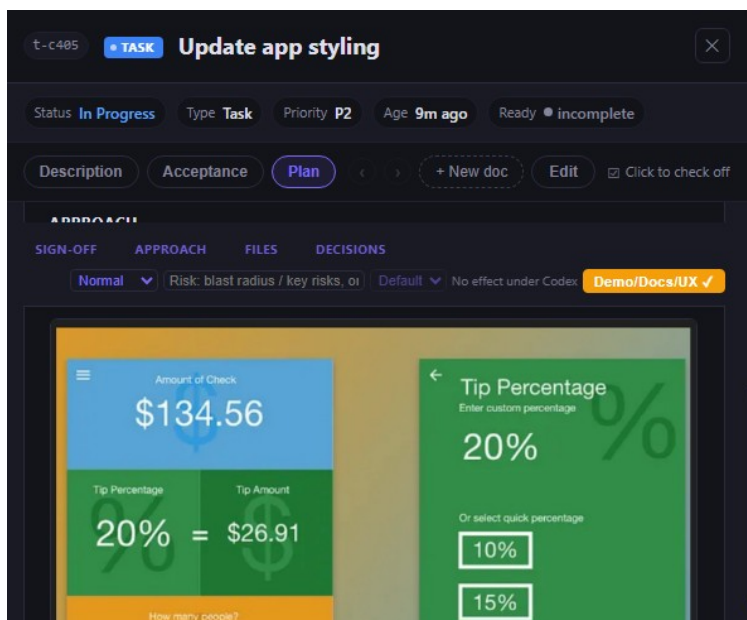
4. Then Click on the ticket on the board, Click Plan and Click Edit



5. Then under the ## Approach section, paste the image. It'll be pasted as a link.

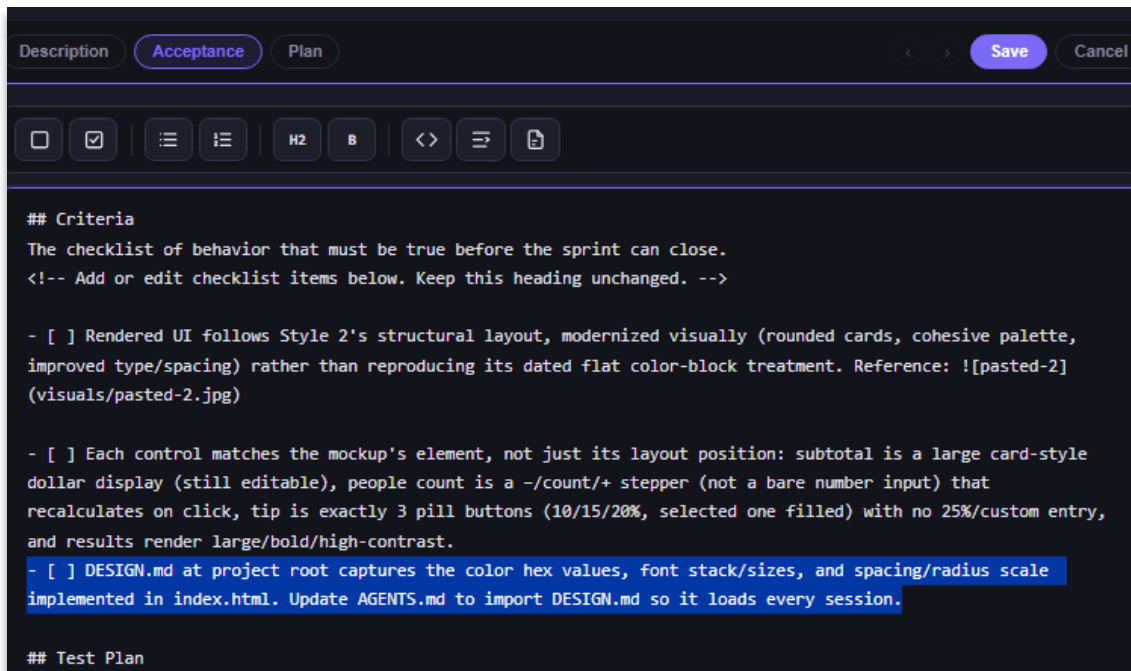


6. Click on **Save** to save the plan. You'll see the image rendered in the Plan now.



7. Let's make our choice **durable**. That way subsequent UI updates can use our new "Design System"
8. Click the **Acceptance** and add this criterion.

DESIGN.md at project root captures the color hex values, font stack/sizes, and spacing/radius scale implemented in index.html. Update AGENTS.md to import DESIGN.md so it loads every session.



The screenshot shows a markdown editor with a dark theme. At the top, there are tabs for 'Description', 'Acceptance' (which is selected), and 'Plan'. To the right of these tabs are 'Save' and 'Cancel' buttons. Below the tabs is a toolbar with icons for list creation, bold text, heading levels (H2, B), and other editing tools. The main text area contains the following markdown:

```
## Criteria
The checklist of behavior that must be true before the sprint can close.
<!-- Add or edit checklist items below. Keep this heading unchanged. -->

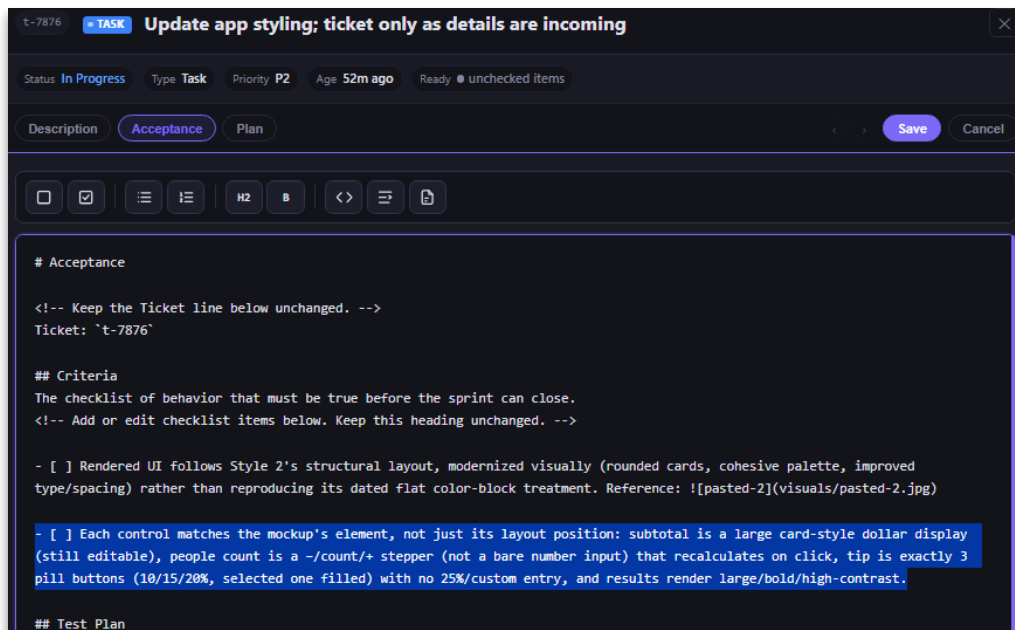
- [ ] Rendered UI follows Style 2's structural layout, modernized visually (rounded cards, cohesive palette, improved type/spacing) rather than reproducing its dated flat color-block treatment. Reference: ![pasted-2](visuals/pasted-2.jpg)

- [ ] Each control matches the mockup's element, not just its layout position: subtotal is a large card-style dollar display (still editable), people count is a ~/count/+ stepper (not a bare number input) that recalculates on click, tip is exactly 3 pill buttons (10/15/20%, selected one filled) with no 25%/custom entry, and results render large/bold/high-contrast.
- [ ] DESIGN.md at project root captures the color hex values, font stack/sizes, and spacing/radius scale implemented in index.html. Update AGENTS.md to import DESIGN.md so it loads every session.

## Test Plan
```

9. **Optional:** If you want the agent to closely follow the UI styling, you can **Edit** the Acceptance criteria and add this criterion:

Each control matches the mockup's element, not just its layout position: subtotal is a large card-style dollar display (still editable), people count is a ~/count/+ stepper (not a bare number input) that recalculates on click, tip is exactly 3 pill buttons (10/15/20%, selected one filled) with no 25%/custom entry, and results render large/bold/high-contrast.



The screenshot shows a task card titled 'Update app styling: ticket only as details are incoming'. The card has a status of 'In Progress', type of 'Task', priority of 'P2', and age of '52m ago'. It also shows 'Ready' and 'unchecked items'. Below the task information are tabs for 'Description', 'Acceptance' (selected), and 'Plan'. To the right are 'Save' and 'Cancel' buttons. The main text area contains the following markdown:

```
# Acceptance

<!-- Keep the Ticket line below unchanged. -->
Ticket: `t-7876`

## Criteria
The checklist of behavior that must be true before the sprint can close.
<!-- Add or edit checklist items below. Keep this heading unchanged. -->

- [ ] Rendered UI follows Style 2's structural layout, modernized visually (rounded cards, cohesive palette, improved type/spacing) rather than reproducing its dated flat color-block treatment. Reference: ![pasted-2](visuals/pasted-2.jpg)

- [ ] Each control matches the mockup's element, not just its layout position: subtotal is a large card-style dollar display (still editable), people count is a ~/count/+ stepper (not a bare number input) that recalculates on click, tip is exactly 3 pill buttons (10/15/20%, selected one filled) with no 25%/custom entry, and results render large/bold/high-contrast.

## Test Plan
```

10. Tell the Agent to go with choice in the Acceptance form. **Note:** that just filling in Criteria, earlier, doesn't substitute for the approval step. Per the **sprint** protocol, planning-file content (step 4) and the approval gate (step 11) are separate.

I've updated ticket plan with a UI style to implement and a new acceptance criteria. Update relevant docs and start the work.

I've updated t-c405 plan with a UI style to implement and a new acceptance criteria. Update relevant docs and start the work.

+

📄

⚡ Auto

↑

11. Once the Agent finishes processing, refresh/reload the **Index.html** in Edge browser to see the **new UI**.

MealSplit

Amount of Check

\$21

Tip Percentage

10%

15%

20%

Tip Amount

\$2.10

How many people?

-

4

+

Comments

It was a ****great value****

Apply

Receipt

Tip: \$2.10

Total: \$23.10

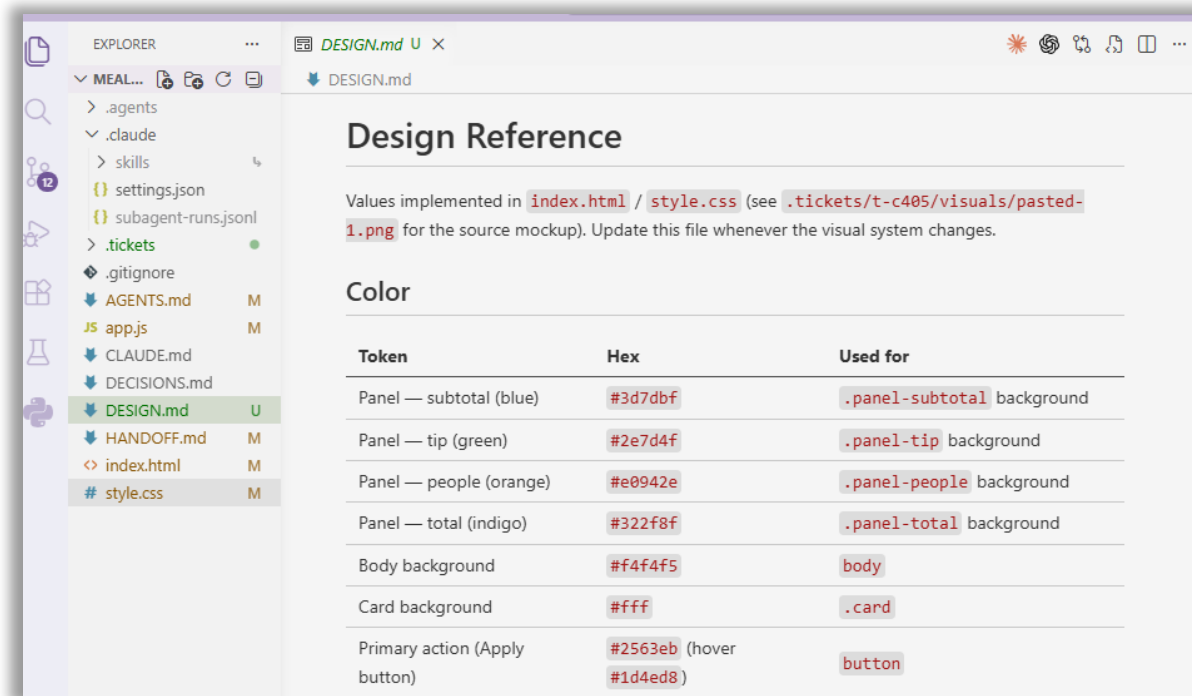
Total Due Per Person

2 people pay \$5.78

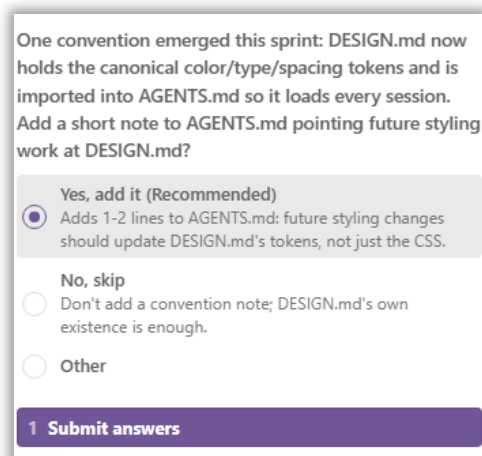
2 people pay \$5.77

It was a **great value**

12. Also, you'll see a new file, **DESIGN.md**, that has all the design information. The **AGENTS.md** now has a reference (DESIGN.md) to incl. this file for any future session.



13. The Agent will ask you to include the newly created DESIGN.md into AGENTS.md since it figures out that you may need it for future UI updates. So, you can go ahead and accept it.



14. **NOTE:**

- The included DESIGN.md in AGENTS.md is enough for **awareness** — since AGENTS.md itself gets injected at every session start (that's how CLAUDE.md → @AGENTS.md already works for this project), DESIGN.md's content now loads automatically too.
- However, to **use this ref**, when you create a new ticket for a new sprint work, you need to be **explicit** in the **Acceptance criteria** to use this design system. Something like this:

- [] New UI reuses DESIGN.md's existing colors/fonts/spacing values — no new hardcoded hex/font-size/spacing values introduced without updating DESIGN.md first.

Session 3 - Running non-interactive canon cli

Session 1 ran the gates and interactively – inside a live chat as part of “sprint complete”. This session runs those same gates *non-interactively*, from a plain terminal with one

This is same mechanism a CI pipeline would run against a pull request (see docs/headless-cli.md). Here you can run locally, so no remote repository is needed.

We’ve 2 options for CLI:

- A. **sprint-headless-eval** – Uses **git diff** static analysis only to check if code was added/updated.
- B. **sprint-headless** – A full 3-gate ceremony with reviewer, evaluator and security-review wrap-up gate.

Command	Runs npm test?	Why
<code>sprint-headless-eval t-oezl --base-ref ...</code>	No	The bash script whitelists Bash to <code>git diff/log/show/status</code> only. Static grading against the diff – npm being installed is irrelevant. Runs via Git Bash (the <code>.cmd</code> finds <code>bash.exe</code>).
<code>sprint complete</code> (interactive close)	Yes	The evaluator subagent gets Read + Bash; on your machine it can actually run your <code>npm test</code> and grade by exit code.
CI gate: eval workflow	No / N/A	The generated <code>canon-gate.yml</code> runs on <code>ubuntu-latest</code> , not your Windows box, and is git-only Bash anyway.

1. Do this only if **git status** shows any changes.

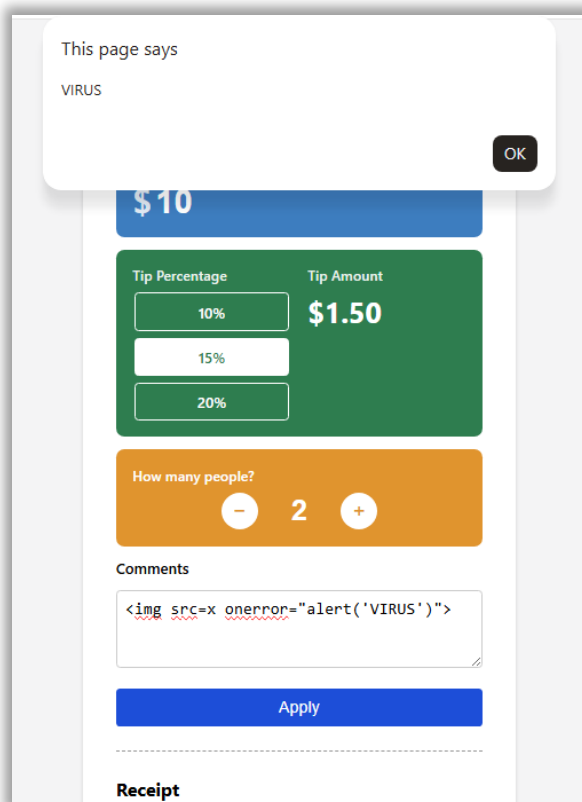
- a. Create a Starting state by committing the last session. Type these commands in the Terminal window

git add . <ENTER>

git commit -m “chore: initial commit” <ENTER>

```
PS C:\Users\agentops\Documents\MealSplit> git add .
warning: in the working copy of '.tickets/ACTIVE', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.tickets/t-6425/acceptance.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.tickets/t-6425/plan.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.tickets/t-6425/research.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.tickets/t-6425/ticket.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'AGENTS.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'CLAUDE.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'DECISIONS.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'HANDOFF.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'calc.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'styles.css', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\agentops\Documents\MealSplit> git commit -m 'chore: initial commit'
[master (root-commit) ac6dcac] chore: initial commit
15 files changed, 382 insertions(+)
create mode 100644 .claude/settings.json
create mode 100644 .gitignore
```


2. Git terms to be aware of:
 - a. main – the default branch; think of it as the ‘last known-good version’
 - b. commit – a saved snapshot of your project’s files, like a checkpoint
 - c. HEAD – whatever commit you’re currently on (your last checkpoint)
3. We’re going to create a new ticket to check the **XSS vulnerability** from Terminal by using the sprint-headless-eval CLI too.
4. **Note:** The **sprint-headless-eval** cannot run anything (npm test , for e.g.) - it grades statically. So for that path the value comes from the evaluator finding the test and the fix in the diff, not from executing them.
5. You can test the app by adding a `<script></script>` text in the Comments box. It will execute the script – cross-site scripting injection (XSS)!



6. Click New Ticket on the Kanban board and add this under in the Acceptance form. In this case we’re manually creating a ticket.
7. In title type ‘XSS Issue and fill the description as shown below. Make **sure** to toggle the **CI**

New Ticket

Create → Cancel

XSS Issue

Detected: Task — general task

Type

Feature Task Bug Chore Epic

Priority

P0 P1 P2 P3 P4

Options

CI Eval-only Eval Override Demo/Docs/UX

H2

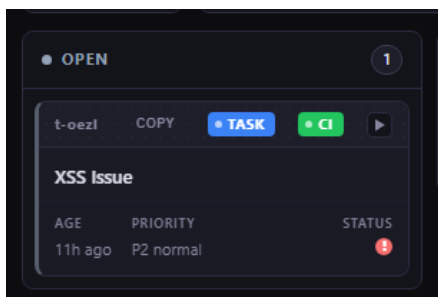
B

<>

Description

Run sprint-headless-eval from CMD line

8. The ticket should look like this on Kanban board



9. Click on the ticket. Then click on **New doc**, select Acceptance and click on **Edit Acceptance**

t-oezl TASK XSS Issue

×

Status Open

Type Task

Priority P2

Age 11h ago

Ready ● needs acc

1 SET BASE REF

2 GRADING IN PROGRESS

3 RESULT READY

Click a step for details

Grades the diff between this commit/branch and your current code — e.g. `main` or a commit SHA.

HEAD~1

▶ Start

◀ ▶

+ New doc

Edit

This ticket is Open — choose a doc type to add:

Acceptance SUGGESTED

Criteria and Test Plan sections both need checklist items — sprint complete blocks without them.

Plan

Capture the approach and record decisions before source edits begin.

Research

Objective compression of truth — what the system does, which files matter, constraints, and unknowns. No implementation planning here.

Edit Acceptance

Cancel

10. Add a criterion and a test plan like this:

For Criteria

Escape the comment text before inserting (e.g. set via a text node / textContent) to disallow script execution.

For Test Plan (input, action, expected observation that an evaluator (or a human) can run and get a yes/no)

Given a comment body `<script>alert(1)</script>`, render it, then assert the output contains the escaped text (`<script>`) as a text node and that no `<script>` element appears in the resulting DOM (i.e. the script does not execute).

Status

In Progress

Type Task

Priority P2

Age 12h ago

Ready • unchecked items

1

SET BASE REF

2

GRADING IN PROGRESS

3

RESULT READY

Click a step for details

Grades the diff between this commit/branch and your current code — e.g. `main` or a commit SHA.

HEAD~1

▶ Start

Description

Acceptance

Eval Report

<

>

Save

Cancel

☑

☰

☰

H2

B

<>

≡

📄

Keep the ticket line below unchanged.

Ticket: `t-oezl`

Criteria

The checklist of behavior that must be true before the sprint can close.

<!-- Add or edit checklist items below. Keep this heading unchanged. -->

- [] Escape the comment text before inserting (e.g. set via a text node / textContent) to disallow script execution.

Test Plan

The commands or checks that prove the criteria work.

<!-- Add or edit test commands below. Keep this heading unchanged. -->

- [x] Given a comment body <script>alert(1)</script>, render it, then assert the output contains the escaped text (<script>...) as a text node and that no <script> element appears in the resulting DOM (i.e. the script does not execute).

11. Then Save the form

12. Now commit the ticket. Open a new Terminal window and run these commands:

- git status <ENTER>
- git add . <ENTER>
- git commit -m 'XSS ticket created' <ENTER>

```

PS C:\Users\agentops\Documents\MealSplit> git add .
warning: in the working copy of '.tickets/t-oezl/acceptance.md', LF will be replaced by CRLF the next time
it touches it
warning: in the working copy of '.tickets/t-oezl/ticket.md', LF will be replaced by CRLF the next time Gi
touches it
warning: in the working copy of 'app.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.tickets/ACTIVE', LF will be replaced by CRLF the next time Git touches
warning: in the working copy of '.tickets/t-oezl/eval-report.md', LF will be replaced by CRLF the next ti
Git touches it
warning: in the working copy of 'app.test.js', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\agentops\Documents\MealSplit> git commit 'XSS Ticket Created'
error: pathspec 'XSS Ticket Created' did not match any file(s) known to git
PS C:\Users\agentops\Documents\MealSplit> git commit -m 'XSS Ticket Created'

[pre-commit] WARNING - ticket set to in_progress by direct file edit.
Prefer 'sprint start' to open a sprint properly.

[pre-commit] Before committing, verify:
1. Open/in-progress tickets - should any be closed first?
   t-oezl   in_progress task    p2  XSS Issue
[master 77d8817] XSS Ticket Created
6 files changed, 173 insertions(+), 10 deletions(-)
create mode 100644 .tickets/ACTIVE
create mode 100644 .tickets/t-oezl/eval-report.md
create mode 100644 app.test.js
PS C:\Users\agentops\Documents\MealSplit>

```

13. Now let's run `sprint-headless-eval` cmd against the starting tag (base, step-16). The resultant output should be FAIL since we haven't implemented the XSS injection safeguards yet.

14. Type and enter this cmd. **Replace** `t-oezl` with your ticket name from the board.

sprint-headless-eval.cmd t-oezl --base-ref base <ENTER>

```

PS C:\Users\agentops\Documents\MealSplit> sprint-headless-eval.cmd t-oezl --base-ref base
Found 2 criterion/criteria in acceptance.md.
Dispatching evaluator against diff base..HEAD ...

```

After a few mins, you should see a "FAIL" and this **eval-report.md** created and the output in Terminal

- HEADLESS_VERDICT: PASS → exit 0
- HEADLESS_VERDICT: FAIL → exit 1

One important nuance for CI triage: exit 1 means "FAIL or invocation error" (auth failure, rate limit, dispatch problem) - it fails closed on purpose, so any problem blocks the merge. That's correct for a gate, but it means a red check isn't always "the code is wrong." To tell a genuine criterion FAIL from an infra error, you read the output (stdout / step summary / eval-report.md). So:

- Merge gate → exit code alone is enough.

- Debuggability / triage → keep the output too (it's basically free via \$GITHUB_STEP_SUMMARY).

